Final Year Project Documentation

Medicare : Smart Healthcare Reminder



By

Laiba Shamim	**2022-CWR-231**
Amna Amjad	**2022-CWR-2**
Javeria Batool	**2022-CWR-226**

Under the Supervision of

Mam Tayyiba Naeem

Bachelor of Science in [Information Technology],

Session(2022-2026)

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY (FCIT),

F. G Postgraduate college for Women,

Kashmir Road Rawalpindi

UNIVERSITY OF THE PUNJAB, LAHORE

Smart Healthcare Reminder

A project presented to

University of the Punjab, Lahore

In partial fulfilment

of the requirement for the degree of

Bachelor of Science in Information Technology,

Session (2022-2026)

By

Laiba Shamim	**2022-CWR-231**
Amna Amjad	**087269**
Javeria Batool	**087272**

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY (FCIT),

F. G Post Graduate College for Women

Kashmir Road Rawalpindi

UNIVERSITY OF THE PUNJAB, LAHORE

DECLARATION

We hereby declare that this software , neither the whole nor a part ,was copied from any source .It is further declared that we have developed this software and accompanying report based on our effort . If any part of this project is proven to be copied from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning .

Laiba Shamim [087251]
Signature :

Javeria Batool [087272]
Signature:

Amna Amjad [087269]
Signature:

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2022-2026),the project Smart HealthCare Reminder and Notification App was developed by laiba Shamim(087251) ,Amna Amjad(087269), Javeria Batool (087272) under the supervision of Mam Tayyiba , in my opinion it is fully appropriate in scope and quality for the degree of Bachelors of science in Information Technology.

FYP Advisor : Mam Tayyiba Naeem
Signature:

Signature (Faculty Advisory Committee (FAC)

	FAC1	**FAC2**
NAME	Mam Tayyiba Naeem	
Signature		

Head of FYP Coordination office : Mam Tayyiba Naeem

Signature:

Head of Department, Information Technology : Prof. Lubna Zakia

Signature: Dated :

Executive Summary

Smart Health Reminder is an android based mobile application developed to help patients in managing their medications, health measurements, appointments and daily activities. The application aims to reduce missed doses, forgotten appointments by providing timely reminders and health tracking features. The application offers a complete health management system that allow user to add and manage medication, track daily intake record important health measurement such as blood pressure and sugar levels, schedule doctor appointments. Chatbot will guide user and provide answer of simple health related question. Feedback feature allow user to send suggestion or complaints to the system. A key feature of the system is the integration of doctor and caregiver roles, making it more than just a reminder application. Patients can enter their disease information and send request to doctors. The doctor can then access the patient's complete health profile. Based on this data, the doctor analyzes the patient's condition and provides personalized recommendations such as medication frequency, measurement schedules, activity plans and caregiver support. One of the primary goal of smart healthcare Reminder is to provide an easy to use and accessible healthcare solution. The application is particularly useful for elderly patients, individuals with constant illness. Smart Healthcare Reminder provides a smart, reliable and user-friendly healthcare platform that increase medication adherence, improves health tracking and enhance overall patient self management.

Acknowledgement

We express our gratitude to our Allah Almighty and after that, we are thankful to our families who allowed us to get higher education, and we become capable of creating Something productive. We're also thankful to our advisor and department faculty for their support

Laiba Shamim : Signature
Amna Amjad : Signature
Javeria Batool : Signature

Abbreviation

Table of all abbreviation and acronyms used in your FYP, along with their respective brief description.

IT	Information Technology
FCF	Firebase Cloud Function
FCM	Firebase Cloud Messaging
BP	Blood Pressure
DB	Database
PU	Punjab University
RAM	Random Access Memory
UI	User Interface
MVVM	Model-view-ViewModel

Table of Content

Contents

1.Introduction 15

1.1 Problem statement 16

1.2 Problem Solution 16

1.3 Objectives of Proposed System 16

1.4 Scope 17

1.5 System Components 18

1.5.1 Module 1: User Authentication (End User Mobile App) 18

1.5.2 Module 2: Patient (End User Mobile App) 18

1.5.3 Module 3: Caregiver (End User Mobile App) 19

1.5.4 Module 4: Doctor (End User Mobile App) 19

1.5.5 Module 5: Pharmacist (End User Mobile App) 19

1.5.6 Module 6: Chat (End User Mobile App) 19

1.5.7 Module 7: Payment Feature (End User Mobile App) 19

1.5.8 Module 8: Medication (End User Mobile App) 19

1.5.9 Module 9: Measurement (End User Mobile App) 20

1.5.10 Module 10: Activity (End User Mobile App) 20

1.5.11 Module 11: Medical professional (End User Mobile App) 20

1.5.12 Module 12: Refill medicines (End User Mobile App) 20

1.5.13 Module 13: Measurement tracker (End User Mobile App) 20

1.5.14 Module 14: Pharmacy Delivery Service (End User Mobile App) 20

1.5.15 Module 15: Chatbot Assistance (End User Mobile App) 21

1.5.16 Module 16 : Notification and Reminder 21

1.5.18 Module 18 : Profile section (End User Mobile App) 21

1.5.19 Module 19: Admin 21

1.7 Vision statement 22

1.8 System Limitations and Constraint 22

1.9 Tools and Technologies 23

1.10 Project Deliverables 23

1.11 Project Planning 24

Gantt Chart 24

1.12 Summary 25

2.Analysis 27

2.1 User Classes and Characterization 27

2.2 Requirement Identifying Techniques 27

2.2.1 Interview 27
2.2.2 Ethnography or observation 28
2.3 Use Case model 28
2.4 Use Case Description 31
2.5 Functional Requirements 54
2.6 Non-Functional Requirements 70
2.7 External interface Analysis: 71
2.7.1User Interface Requirements 71
2.7.2 Software Interfaces 71
2.7.3 Hardware Interfaces 71
2.7.4 Communication Interfaces 71
2.7.5 Error handling and Reporting 72
2.8 Summary 72
3.System Design 74
3.1 Design Considerations 74
Assumptions and Dependencies: 74
Limitations 74
3.2 Design Models 74
1:Class Diagram 75
2: Sequence Diagram 76
3:State Diagram 79
3.3 Architectural Design 83
3.3.1 Architecture Pattern 85
3.3.2 Component structure 85
3.3.3 Component Diagram 87
3.4 Data Dictionary 88
3.5 User Interface Design 89
3.5.1 Screen Images 89
3.6 Design Decisions 90
3.7 :Summary 91
4. Implementation 93
4.1 Algorithms 93
4.2 External APIs/SDKs 101
4.3 Code Repository 102
5: Introduction 104
5.1 Unit Testing (UT) 104

5.2 Functional Testing (FT) 112
5.3 Integration Testing (IT) 124
5.4 Performance Testing (PT) 133
Summary 137
6.1 Conversion Method 139
6.1.1 Pilot Conversion 139
6.2 Deployment 139
6.2.1 Deployment steps: 140
6.2.2 Data Conversion 140
6.2.3 Training 141
6.3 Post Deployment Testing 142
6.3.1 Post Deployment Testing Steps: 142
6.4 Challenges and Solutions 143
6.5 Summary 144
7. Introduction 146
7.1 Conclusion 146
7.2 Future Works 146
Appendix A: General Coding Standards & Guidelines 147
1. Variable Naming Conventions 147
2. Visual Scope Indicators 147
3. Read-Only and Immutable Fields 147
4. Function Naming and Behavior 147
5. Commenting Style 148
6. Indentation and Code Structure 148
7. File and Folder Naming 148
8. Widget and Class Structure 148
9. Access Modifiers and Scoping 148
10. Cohesion and Coupling 148
11. DRY Principle (Don't Repeat Yourself) 149
12. Single Responsibility Principle (SRP) 149
13. Extensibility and Maintainability 149
14. Exception Handling and Logging 149

List of Tables

Table 1: Tool and Technology 23
Table 2:User classes and characteristics 27
Table 3 Use Case1 31
Table 4:Use case 2 32
Table 5:Use case 3 33
Table 6:Use case 4 34
Table 7:Use case 5 35
Table 8:Use case 6 37
Table 9:Use case 7 38
Table 10:Use case 8 39
Table 11:Use case 9 40
Table 12:Use case 10 41
Table 13:Use case 11 42
Table 14:Use case 12 43
Table 15:Use case 13 44
Table 16:Use case 14 44
Table 17: Use Case15 45
Table 18:Use Case 16 46
Table 19 :Use Case 17 47
Table 20:Use Case 18 48
Table 21:Use Case 19 49
Table 22:Use Case 20 50
Table 23 :Use Case 21 51
Table 24:Use Case 22 52
Table 25:Use Case 23 53
Table 26:Use Case 24 53
Table 27:Functional Requirement 1 54
Table 28:Functional Requirement 2 55
Table 29:Functional Requirement 3 56
Table 30:Functional Requirement 4 57
Table 31:Functional Requirement 5 57
Table 32:Functional Requirement 6 58
Table 33:Functional Requirement 7 59
Table 34:Functional Requirement 8 60
Table 35:Functional Requirement 9 60
Table 36:Functional Requirement 10 61
Table 37:Functional Requirement 11 62
Table 38:Functional Requirement 12 62
Table 39:Functional Requirement 13 63
Table 40:Functional Requirement 14 64
Table 41:Functional Requirement 15 64
Table 42:Functional Requirement 16 65
Table 43:Functional Requirement 17 65
Table 44:Functional Requirement 18 66

Table 45:Functional Requirement 19 67
Table 46:Functional Requirement 20 67
Table 47:Functional Requirement 21 68
Table 48:Functional Requirement 22 68
Table 49:Functional Requirement 23 69
Table 50:Functional Requirement 24 70
Table 51:Unit Test Case 1 104
Table 52:Unit Test Case 2 105
Table 53:Unit Test Case 3 105
Table 54:Unit Test Case 4 106
Table 55:Unit Test Case 5 107
Table 56:Unit Test Case 6 108
Table 57:Unit Test Case 7 109
Table 58:Unit Test Case 8 110
Table 59:Unit Test Case 9 110
Table 60:Unit Test Case 10 111
Table 61:Functional Test Case 1 112
Table 62:Functional Test Case 2 113
Table 63:Functional Test Case 3 114
Table 64:Functional Test Case 4 115
Table 65:Functional Test Case 5 116
Table 66:Functional Test Case 6 117
Table 67:Functional Test Case 7 118
Table 68:Functional Test Case 8 119
Table 69:Functional Test Case 9 120
Table 70:Functional Test Case 10 121
Table 71:Functional Test Case 11 122
Table 72:Functional Test Case 11 123
Table 73:Integration Test Case 1 124
Table 74:Integration Test Case 2 125
Table 75:Integration Test Case 3 126
Table 76:Integration Test Case 4 127
Table 77:Integration Test Case 5 128
Table 78:Integration Test Case 6 129
Table 79:Integration Test Case 7 130
Table 80:Integration Test Case 8 131
Table 81:Integration Test Case 9 132
Table 82:Performance Test Case 1 133
Table 83:Performance Test Case 2 134
Table 84:Performance Test Case 3 135
Table 85:Performance Test Case 4 135
Table 86:Performance Test Case 5 136
Table 87:Performance Test Case 6 137

List of Figure

Figure 1:Use Case Diagram 28
Figure 2:Class Diagram 75
Figure 3:Sequence Diagram 76
Figure 4:State Diagram 79
Figure 5:Architecture Diagram 85
Figure 6:Component Diagram 87

Chapter 1
Introduction

1.Introduction

In today's digital world maintaining a healthy lifestyle has become increasingly challenging. Many users rely on paper notes or memory which can lead to missed medications or appointments. Health information is usually kept in different places and not saved in one organized system. In many cases people's memory becomes weaker as they get older. In earlier times, people only arranged their pills physically in boxes. There were no automatic reminders no data storage no tracking system and no proper way to manage overall health information. Many Applications are already available but most of them are not user-friendly, for example: Pill reminder, Dosecast, Medisafe. These Applications provide only medicine reminder, user cannot track their full health routine at one platform. This Application brings the trend of mobile based health management systems along with the problem to patients who missed their medicine dose and other regular routines. The purpose of this application is to manage health through technology and make a person's medical life easier. As healthcare needs became more complex, users required a system that could integrate multiple types of reminders in one application. Application Smart Healthcare Reminder allow users to maintain medication schedule, record health measurement, manage daily activities, manage appointments. A key feature of the system is the integration of doctor and caregiver roles making it more than just a reminder application. Patients can enter their disease information and send request to doctors. The doctor can then access the patient's complete health profile. Based on this data, the doctor analyzes the patient's condition and provides personalized recommendations such as medication frequency, measurement schedules, activity plans. The doctor can evaluate whether the patient is capable of managing their health independently or requires additional support. If necessary, the system allows the involvement of a caregiver who can manage healthcare activities on behalf of the patient. Depending on the patient's condition:

- The patient may manage tasks independently
- The caregiver may manage tasks
- Or both patient and caregiver can collaboratively manage the healthcare routine

The system also includes an AI chatbot that provides intelligent health guidance by responding to user queries offering basic medical advice and assisting users in managing their daily healthcare activities. The system will send notification at schedule time. The doctor and patient can communicate through their respective dashboards to discuss recommendations, health-related concerns. Emergency contact allow user to save emergency contact details. The patient can quickly access this number in case of an emergency. The system is designed to be flexible and scalable which means it can be used by different users with different needs. The application aims to fulfill these needs with interacting features with the help of several tools and technologies along with constraints, limitations.

1.1 Problem statement

In today ’s era people has shortage of time ,busy routine or due to multiple health issues they cannot manage their medical routine. According to studies, between 40% and 75% of adults fail to take their prescribed medication on time each day. As a result , individuals fail to take the right dosage at the right time which may result in additional patient suffering and put their lives in danger. Many patients cannot manage their daily routine which is essential for managing their health. There is a lack of coordination between patients, doctors and caregivers. Doctors do not have access to patient adherence data making it difficult to analyze patient conditions accurately. Similarly, patients who are critically ill or elderly often require assistance but do not have a proper system to involve caregivers effectively. This application Smart Healthcare Reminder Will work as intelligent mobile-based health assistant which help patients to manage their daily health routine.

1.2 Problem Solution

Many reminder Applications are available in market but they allow user to enter the medicine name, time or quantity or give notification at that time but this application fulfill this gap and provides all-in-one solutions and not only provide medicine reminder but also manages patients daily routine such as Measurement(Bp ,sugar, Weight),Activity(Meal, water Intake, Walking, Exercise, Yoga, Cycling),Appointment and Refill. Appointment module will help user to remind their medical checkup. Refill module will help to restock their medicine when stock will be limited. Chatbot will guide user and provide answers to simple health related questions and allow users to give feedback which can be view by admin. A key enhancement of the system is the integration of doctor and caregiver roles. Patients can send request to doctors who can then access the patient’s complete health profile, including reminder adherence and measurement history. Based on this data the doctor can analyze the patient’s condition and provide personalized recommendations, such as medication adjustments, monitoring frequency, and the need for caregiver support. After the doctor approves the patient's request, a chat option becomes available for both the doctor and patient and its caregiver if doctor give recommendation of caregiver to patient. The patient and caregiver can also communicate through their respective dashboard. The doctor and patient can communicate through their respective dashboards to discuss recommendations, health-related concerns, and follow-up information.

1.3 Objectives of Proposed System

1. The user will authenticate system with name, email ,password ,Date of Birth and gender
2. To never miss a dose, allow application “Medicare” to send you notification.
3. Patient section allows users to send request, view doctor recommendations and manage reminders based on their health plan.

4. Doctor section allows doctors to review patient profiles, analyze health data and provide recommendations.
5. After submitting a recommendation, the doctor can provide their Easypaisa or JazzCash number to the relevant patient through the chat. The patient can access the Pay Doctor option under the doctor section, enter the doctor's name and payment number and submit the transaction ID along with a payment receipt screenshot.
6. The doctor approves the patient's request, a chat option becomes available for both the doctor and patient.
7. Caregiver section allows caregiver to monitor patient data, manage health tasks, update reminders and assist the patient in following their health plan and can also communicate with patient.
8. Medication section add medication to your treatment plan to get reminders.
9. Measurement section helps to measure health data like blood pressure , weight and sugar. Measurement section helps to measure health data like blood pressure , weight and sugar. Setup reminders for daily habits such as walking, drinking water, meal or exercise, yoga or cycling.
10. Appointment module will help user to remind their medical checkup.
11. Pharmacy Medicine Delivery allows patients to order required medicines, check availability, and receive them at their specified delivery address.
12. Chatbot will guide users and give basic health tips
13. Refill module will help to restock their medicine.
14. Notification will be send based on the reminder time set by the user in different module.
15. Profile section to view intake progress stored in history section and feedback option will allow user to share their experience and complication.
16. Emergency contact allow user to save emergency contact details. The patient can quickly access this number in case of an emergency.

1.4 Scope

The scope of the Smart Healthcare Reminder system is to develop a mobile application that helps users manage their daily health activities in an organized and simple way. Patients can manage their health routines and follow doctor recommendations. Doctors can access patient profiles after appointments, analyze health data, and provide recommendations such as treatment plans, monitoring frequency, and caregiver requirements. Caregivers can assist patients by managing tasks and monitoring their health when needed. The application will allow user to set reminders for medicines ,health measurement(blood pressure, sugar level, weight),physical activities(walking, drinking water, eating meal, exercise, yoga, cycling),medical appointments , medicine refill and includes chatbot that guide users and answer simple health questions .Pharmacy Medicine Delivery allows patients to order required medicines check

availability and receive them at their specified delivery address. Allow users to give feedback which can be viewed by admin. It will send notification at scheduled time and allow users to mark tasks as Done or Dismiss(mark by patient/caregiver). Measurement tracker allow users to record their values(BP(systolic/diastolic),sugar level, weight)and consistently monitor them. The system will also store daily records so user can track their health progress over time. This application is develop for individuals who want to manage their daily health routine and avoid forgetting important health tasks. Emergency contact allow user to save emergency contact details . The main goal of the project is to offer a simple, easy to use and reliable health reminder application where users can manage their health tasks at one place.

1.5 System Components

The Application is divided into main parts

1. Patient
2. Caregiver
3. Doctor
4. Pharmacist
5. Admin

The Application will work in online mode. When the user has an active internet connection, it will allow users to manage Medication, Measurement, Appointments, Activities, Refill and Profile information. All updates, notifications, and data backup are handled online.

1.5.1 Module 1: User Authentication (End User Mobile App)

The list of features this module will have and how it will be used are the following.

1) A splash screen to initialize or load user specific data if authenticated, else redirected to authorization screen. The splash screen will check if there's a user logged in or not using Firebase Authentication. If User is logged in then user will be redirected to home screen.

2)The authorization page will have the following option log In and Sign UP.

3)The Sign-Up screen will ask the user to enter his personal details like email, full name, date of birth, gender, and password.

4)The Log In screen will ask the user to enter the registered email and password. After validation, the user will be redirected to the home screen. Login screen will also have options for forgotten password with email verification.

1.5.2 Module 2: Patient (End User Mobile App)

Patient Module allows users to manage their daily health activities such as setting medication reminders, tracking measurements (BP, sugar, weight), and managing daily routines. Patients can send request to doctors and follow their recommendations.

1.5.3 Module 3: Caregiver (End User Mobile App)

Caregiver Module allows caregivers to monitor patient health data and manage healthcare tasks on behalf of the patient. Caregivers can view reminders, update task status and track patient progress. This module helps ensure continuous care when the patient is unavailable or unable to manage their routine. The caregiver approves the patient's request chat option becomes available for both the caregiver and patient. The patient and caregiver can communicate through their respective dashboard.

1.5.4 Module 4: Doctor (End User Mobile App)

Doctor Module allows doctors to view patient profiles after accepting the request and analyze their health data, including measurements and reminder history. Doctors can provide recommendations such as medication plans, monitoring frequency. The module also helps doctors decide whether the patient requires caregiver support or can manage independently or can manage both patient and caregiver and needs physical caregiver support or not.

1.5.5 Module 5: Pharmacist (End User Mobile App)

The pharmacist receives the request through the pharmacy Dashboard and provides the medicine that are in Stock and delivery cost. The pharmacist then reviews and accepts the delivery address to proceed with the medicine delivery.

1.5.6 Module 6: Chat (End User Mobile App)

The doctor approves the patient's request, a chat option becomes available for both the doctor and patient. The doctor and patient can communicate through their respective dashboards to discuss recommendations, health-related concerns. The caregiver approves the patient request and a chat option becomes available for both the caregiver and patient. When doctor give recommendation of patient and caregiver support then caregiver can also communicate with doctor in private chat and can do group chat.

1.5.7 Module 7: Payment Feature (End User Mobile App)

After submitting a recommendation, the doctor can provide their Easy paisa or Jazz Cash number to the relevant patient through the chat. The patient can access the Pay Doctor option under the doctor section, enter the doctor's name and payment number and submit the transaction ID along with a payment receipt screenshot. The payment remains pending until the doctor reviews it. The doctor can then Approve or Reject the payment and the updated payment status is displayed to the patient.

1.5.8 Module 8: Medication (End User Mobile App)

The user can add a new medication by entering details such as medicine name, dosage , frequency ,start date, end date and reminder time. After saving the information, the medication is added to reminder list. Once a medication is added, the system automatically gives reminders based on the specified time. Upon receiving a reminder, the user can mark the medication status as taken or missed. The user can remove a medication from the list when it is no longer required. Deleting a medication cancels its reminders.

1.5.9 Module 9: Measurement (End User Mobile App)

The user can set reminders for health measurements such as blood pressure, sugar level, body weight by selecting the measurement type and preferred reminder time. At the predefined time, the system sends a notification reminding the user to take the selected health measurement. The user can view a list of all active measurement reminders. The user can update reminder details such as the measurement type or reminder time. The user can delete a measurement reminder when it is no longer needed.

1.5.10 Module 10: Activity (End User Mobile App)

The user can set reminders for daily activities such as drinking water, eating meals, walking or exercise, yoga or cycling by selecting an activity type and preferred reminder time. The user can choose the frequency of reminders, including daily or multiple times per day based on their routine and health goals. At the scheduled time, the system sends notification reminding the user to perform the selected Activity. The user can view a list of all active Activity reminders from the Activity section. The user can update Activity reminder details such as reminder or frequency. The user can delete an activity reminder when it is no longer needed.

1.5.11 Module 11: Medical professional (End User Mobile App)

The Medical Professional module allows patients to connect with doctors and caregivers and manage appointments. User can send doctor connection requests with their symptoms or invite caregivers using their name and email. User can also switch to the Appointments tab to view and schedule medical visits with reminder alerts.

1.5.12 Module 12: Refill medicines (End User Mobile App)

The user can set refill reminders for their medicines by entering details such as medicine name or stock in medication section. The system monitors the refill schedule. This feature helps users avoid running out of essential medicines.

1.5.13 Module 13: Measurement tracker (End User Mobile App)

The user can record and track their health measurements such as blood pressure, blood sugar, and body weight. The user can enter measurement values like systolic and diastolic for blood pressure, sugar level for blood sugar, and weight . The system saves the recorded measurements so the user can view them later. The user can see a list of all previous measurement records to monitor their health condition. The user can also update or delete a measurement record if it was entered incorrect.

1.5.14 Module 14: Pharmacy Delivery Service (End User Mobile App)

Pharmacy Delivery Service allows patients to search for their required medicines, enter the medicine name and quantity and check their availability for delivery by pharmacist.The patient can provide the delivery address and place the medicine order when the required medicine is available. The pharmacist

calculates the medicine and delivery cost, saves the order details and displays an order confirmation to the patient.

1.5.15 Module 15: Chatbot Assistance (End User Mobile App)

The chatbot works as an assistant that helps users with basic health-related questions. The user can ask questions about reminders or general health tips. The chatbot provides simple responses. It can also suggest actions such as checking blood pressure, drinking water, or doing light exercise when health values are high or low. This feature helps users quickly get helpful information.

1.5.16 Module 16 : Notification and Reminder

The system automatically generates and schedules notification based on the reminder time set by the user in different modules. At the scheduled time, the user receives a notification reminding them of specific task .Each reminder must give only one notification when user marked it as Done or Dismiss(mark by patient/caregiver).When users sends and receive message notification will also pop-up reminding them of the message.

1.5.17 Module 17: Track Intake (End User Mobile App)

The track intake module tracks the medicine intake also measurement (BP, Sugar, weight),Daily activities ,Appointment. The purpose of this module is to ensure that the user can confirm and review everything related to their health routine in one place .When a reminder notification appear the user can update its status by selecting options such as Done or missed.

1.5.18 Module 18 : Profile section (End User Mobile App)

The profile section includes history option which allows users to view the summary of health-related progress Users can submit feedback in the feedback option and also include an emergency contact option to add the name or phone number.

1.5.19 Module 19: Admin

The admin can view registered users. Verify the doctor, pharmacist and nurse license. User can submit feedback which can be viewed by the admin.

1.6 Related System Analysis /Literature Review

With the increased use of smartphones and mobile-health technology, many reminder applications have entered the market. Most of the applications currently available primarily focus only on medicine reminders. This proposed application aims to fill this gap by providing an all-in-one solution. The proposed application not only provide medicine reminders but also helps manage patients daily routine such as meal schedule, water intake, walking routine or exercise. It also includes doctor appointment reminder and medicine refill reminder. The user interface is easy to use and users can

navigate through app with ease . Most existing applications such as Medisafe and Dosecast provide only medicine reminders and do not offer integrated options to manage other aspects of health.

Application Name	Weakness	Proposed Project Solution
Medisafe	Provide only medicine reminder. user cannot track their full health routine at one platform	App manage complete health routine, user will not miss the important health task.
Dose cast	Dose cast give basic refill reminders does not give smart options like medicine quantity tracking or low-stock alert.	Medicare track medicine and send a refill reminder when stock is low.
Pill Reminder	Pill reminder offers a simple alarm based reminder with no intake tracking or health summary.	Application gives intelligent reminders intake tracking(Done, Dismiss) and detail health progress summary.

1.7 Vision statement

To create an intelligent mobile application that help individuals manage their daily health task by providing smart reminder for medicine, measurement, activities, appointments and refill. The application also includes measurement tracker to record and monitor blood pressure, blood sugar, weight and Chatbot that provide guidance, answers basic health question and help user manage reminders and coordinate with doctor, caregiver and pharmacist . Encourage user to stay consistent with their health routine and minimizing the chances of missed or late care. The application aims to stay simple and smart so people of all ages can manage their health easily without any confusion.

1.8 System Limitations and Constraint

Limitation:

Internet dependency: The application needs compatible smartphones and stable internet which may limit some users.

User dependency: Reminder effectiveness depends on user interaction correctly setting reminders and responding to notification.

Notification delays: System or device restrictions can delay notifications.

Constraints:

The system constraints will be:

1. The application is designed only for android devices.
2. The application reminder functionality depends on user allow notification permission.

1.9 Tools and Technologies

Our system use these tools and technologies for complete implementation of goals / objectives :

Table 1: Tool and Technology

Tools /Technology	Version	Rational
Android studio	Panda 2(2025.2.3)	Official IDE for android app development.
Firebase auth	5.4.1	For authentication and registration of users.
Draw.io	24.x	For creating Diagrams like use case, state transition, sequence and class for better visualization.
Flutter	3.38.6	Familiarity with Flutter due to Mobile Application course. Now Application are developed in flutter.
GitHub	Build-in Android studio	Used for version control and hosting project source code online.
Cloudinary	Cloud(API)	Used for cloud storage and securely handling file uploads for the mobile application.
Firebase Firestore	5.6.0	A database for storing user history, chat and reminders.
Flutter local Notification	17.2.4	Local notification are used to remind or alert the user

1.10 Project Deliverables

Our system will have following deliverables by the end of 7th semester:

1. Project Proposal
2. Final Year Project Registration Form

3. Software Requirement Specification (SRS)
4. Software Design Specification (SDS)
5. Prototype
6. Comprehensive Evaluation Document (Proposal, SRS, SDS)

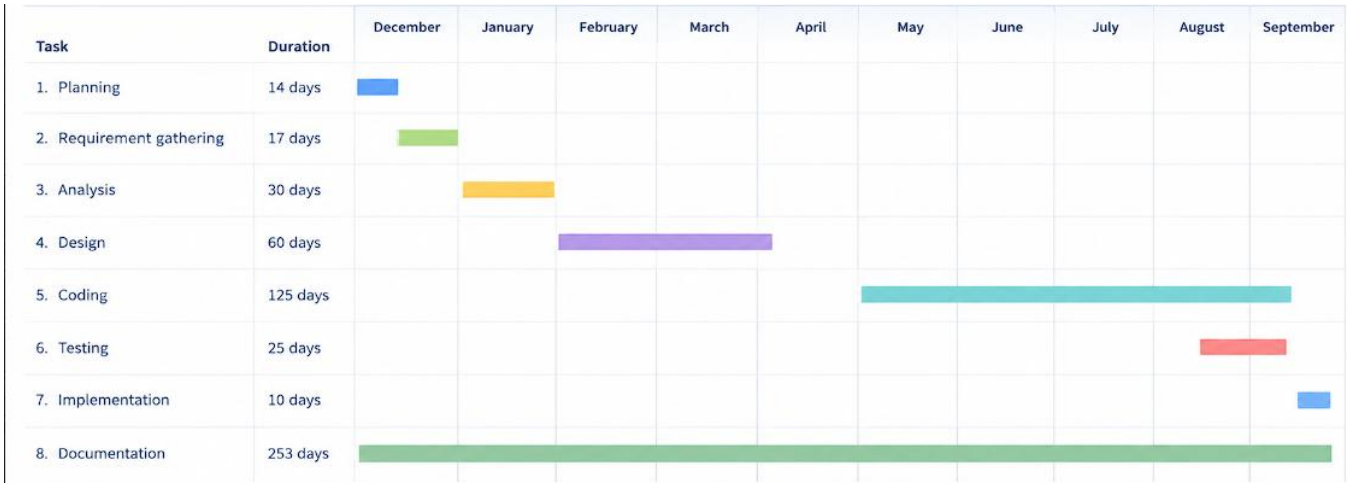
Our system will have the following deliverables by the end of 8th semester:

1. Coding/Implementation
2. Testing
3. Deployment
4. Final
5. Presentation & Final Evaluation

1.11 Project Planning

	Task name	Duration	Start	Finish	Predecessor
1	Planning	14 days	01/12/25	14/12/25	1
2	Requirement gathering	17 days	16/12/25	1/1/26	2
3	Analysis	30 days	2/1/26	31/1/26	2
4	Design	60 days	2/2/26	2/4/26	2
5	Coding	125 days	1/5/26	17/9/26	2
6	Testing	25 days	24/8/26	18/9/26	5
7	Implementation	10 days	19/9/26	28/9/26	6
8	Documentation	253 days	1/12/25	30/9/26	2

Gantt Chart



1.12 Summary

To get together with emerging trend of mobile-health management system and solve patient's problem who forget about their dose and regular routine, our project will deliver its modules and fulfill its goals and objective with the planned time frame while maintaining the scope of project to deliver a quality product.

Chapter 2

Requirements Analysis

2.Analysis

This chapter focuses on requirement identification, elicitation and analysis. The users are categorized into two classes. For requirement identification and elicitation open interviews, ethnography and use case analysis are used. The identified requirements are categorized into functional and non-functional requirements along with their use case description and diagram.

2.1 User Classes and Characterization

Table 2:User classes and characteristics

User class	characterization
Patient	The patient logs into the application to manage medications, health measurements, appointments reminders.
Caregiver	The caregiver is an authorized person who can log in to manage reminders and health tasks on behalf of the patient when the patient is unavailable.
Doctor	The doctor logs into the application to view patient profiles after appointments and analyze their health data . The doctor gives advice on medicine, Measurement ,activity plan and caregiver support.
Pharmacist	The pharmacist receives the request through the Pharmacy Dashboard and provides the medicine and delivery cost. The pharmacist checks and accepts the delivery address to send the medicine.
Admin	The admin can view registered users. Verify the doctor, pharmacist and nurse license. User can submit feedback which can be viewed by the admin.

2.2 Requirement Identifying Techniques

There are fundamental approaches used to identify requirements. As elicitation of requirements requires meeting with stakeholders so following are the approaches used:

2.2.1 Interview

Open interviews were conducted with individuals from different age groups to understand the difficulty they face in managing medicines, appointments and daily health activities. The application helps to identify common problems such as forgetting doses and missing appointments which helps in defining the system requirement.

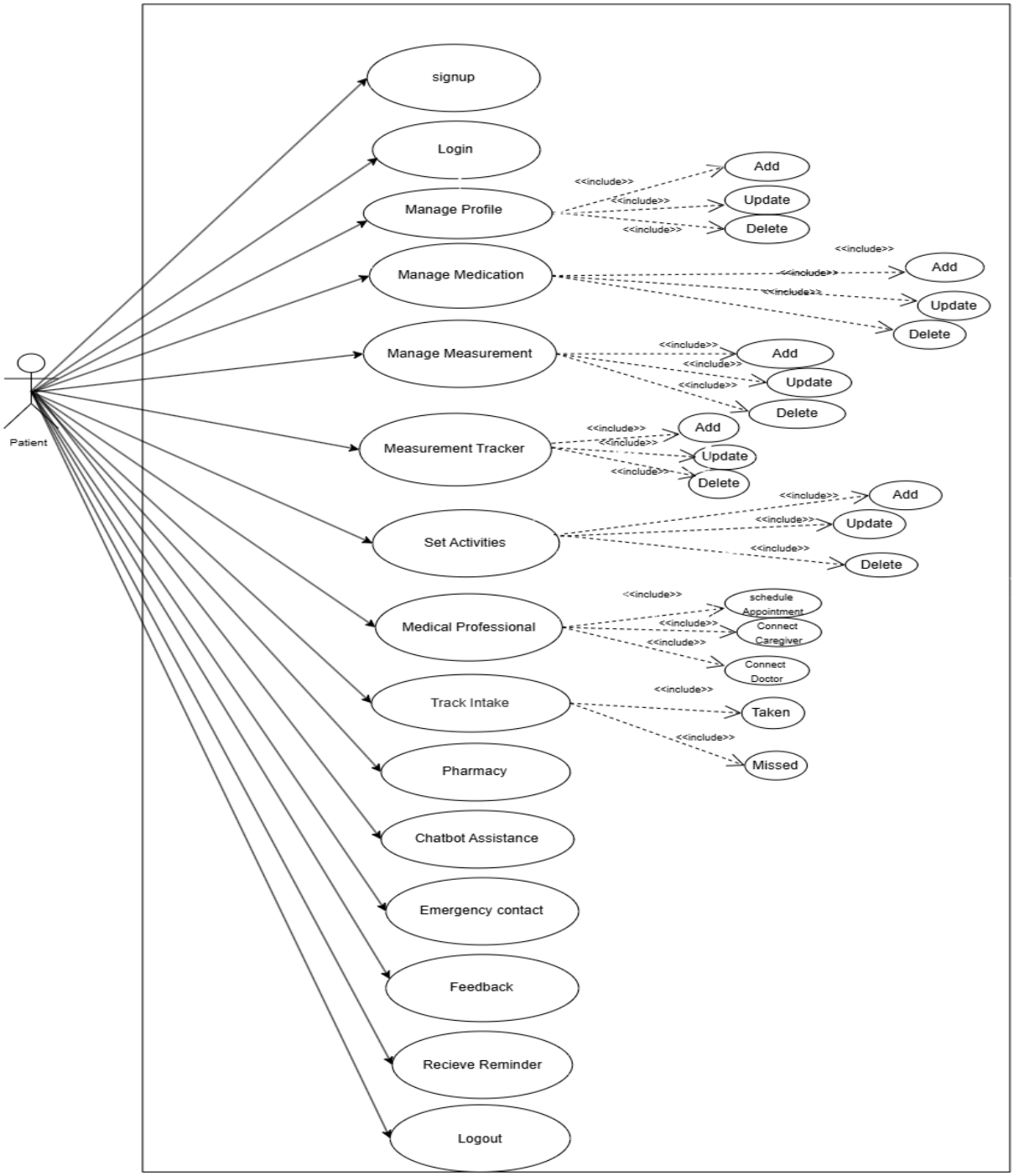
2.2.2 Ethnography or observation

Software systems are not used alone they work in social and organizational environments. User needs are often affected by their daily routines and surroundings. Ethnography is an observational technique that can be used to understand operational processes and help derive requirements for software to support these processes. It was noticed that many people rely on memory or simple alarms and often forget important health tasks. This observation helped in identifying the need for a structured and organized healthcare reminder system

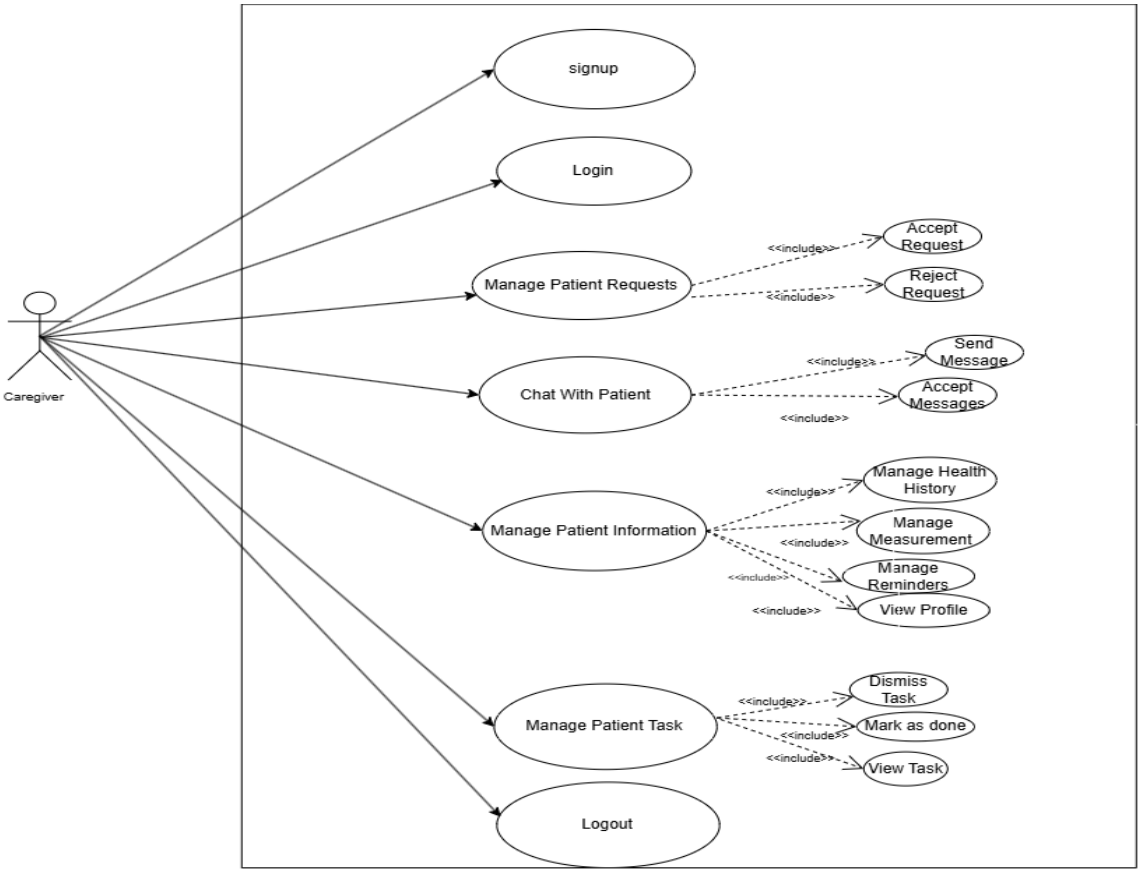
2.3 Use Case model

Figure 1:Use Case Diagram

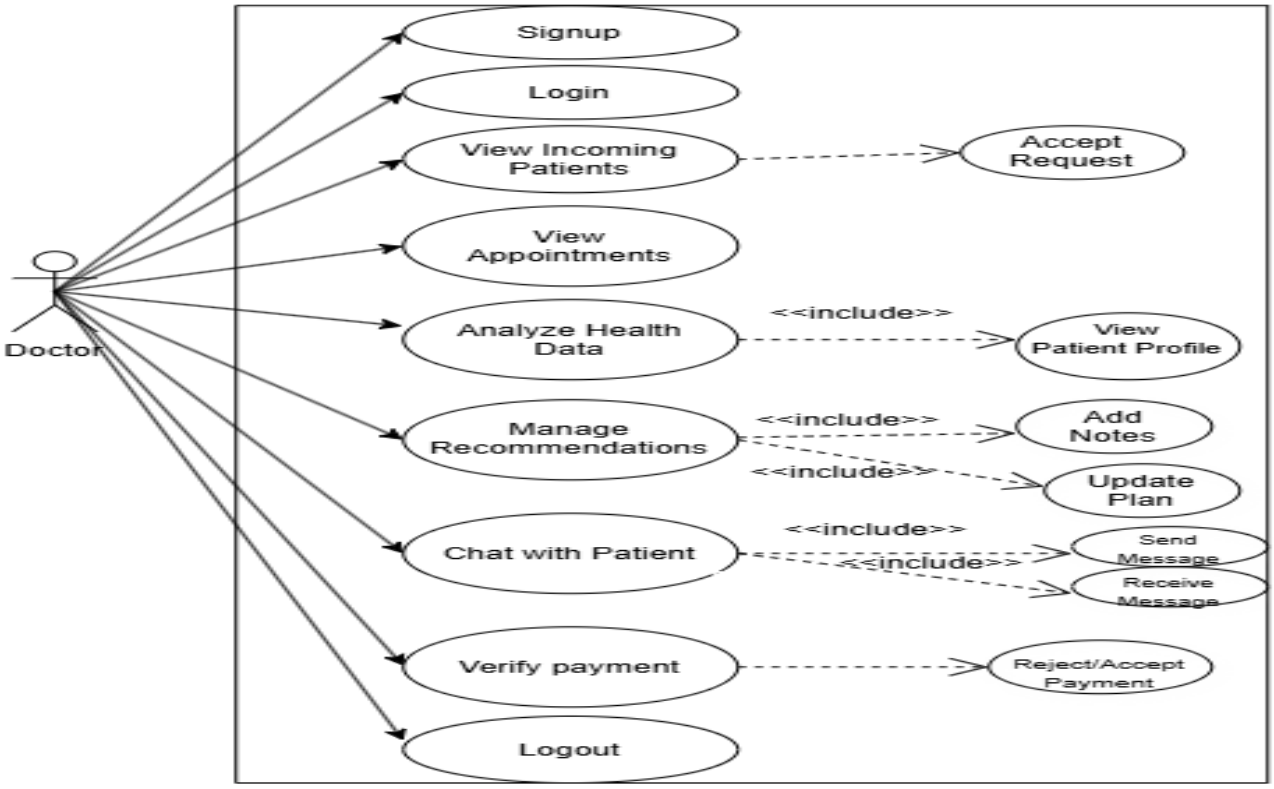
Use Case Diagram 1:



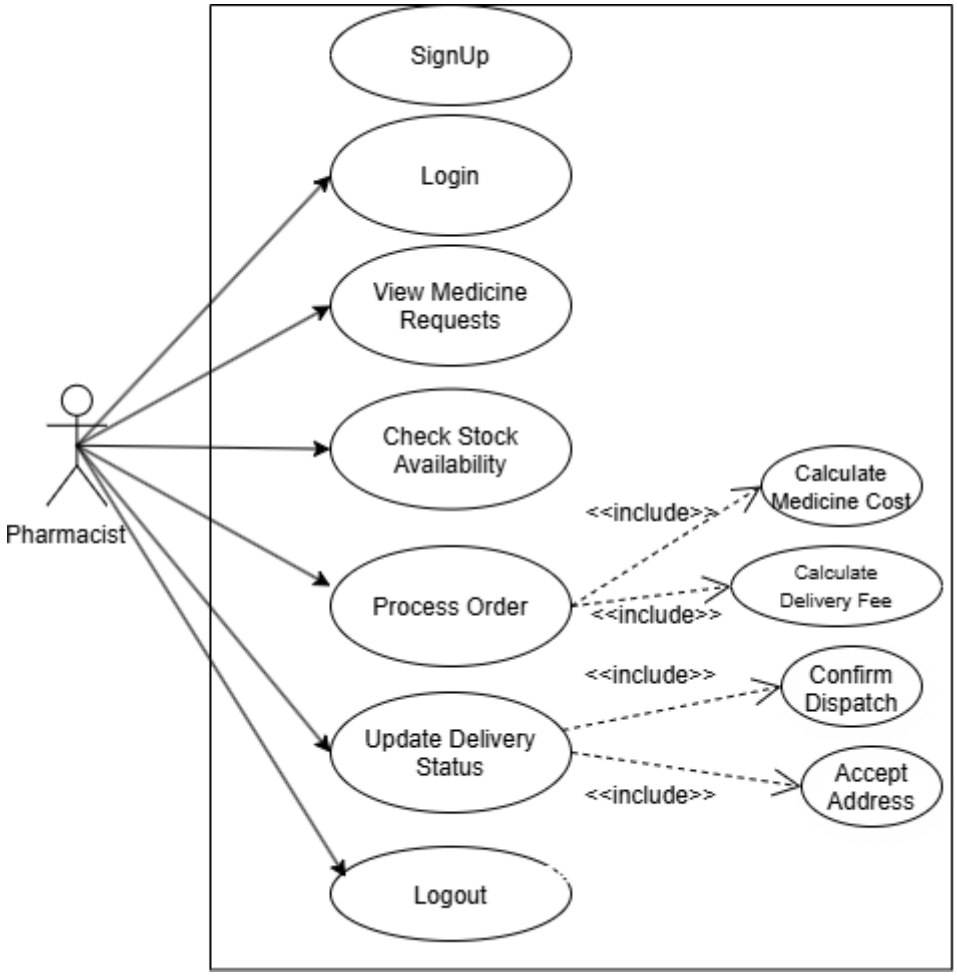
Use Case Diagram 2:



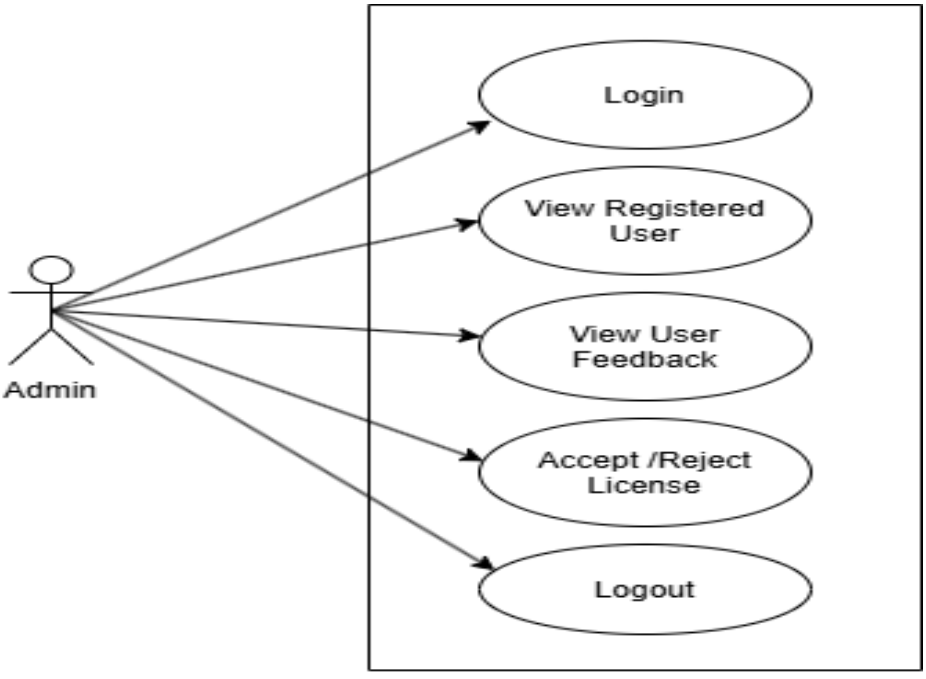
Use Case Diagram 3:



Use Case Diagram 4:



Use Case Diagram 5:



2.4 Use Case Description

Table 3 Use Case1

Use Case 1

Use Case ID	UC-1
Use Case Name	Sign Up with Email and password
Actors	Patient/Caregiver/Doctor/Pharmacist
Description	This use case allows a user to register by providing their personal details.
Trigger	User selects the “sign up” option from the authorization screen.
Preconditions	User must not already be registered.
Postconditions	A new user account is created in firebase and the user is redirected to the profile completion screen.
Normal Flow	1. User opens the application. 2. Authorization screen appears. 3. User selects “sign up”. 4. User enters email, name, DOB ,gender, password and for doctor, pharmacist,caregiver(nurse) they have to enter their registered license. 5. Form is validated. 6. Firebase authentication creates user.
Alternative Flows	If the email is already registered an error message is shown. If password does not meet criteria the user should correct it.
Business Rules	Email must be unique and valid. Password must be at least 8 characters. Required fields cannot be empty.
Assumptions	Firebase authentication is active and integrated. User devices has internet access.

Table 4:Use case 2

Use Case 2

Use Case ID	UC-2
Use Case Name	Login with email and password
Actors	Patient/Caregiver/Doctor /Pharmacist/Admin
Description	This use case allows registered users to login into the application using their email and password and if user forgot password then password will received on registered email.
Trigger	The user opens the application and selects the “login” option on the authorization screen.
Preconditions	The user must already register with a valid email and password.
Postconditions	If the login is successful, the user is redirected to the home screen. If not an error message is shown.
Normal Flow	1. The user opens the application. 2. Splash screen checks the login status. 3. User is not authenticated and is taken to authorization screen. 4. Users select login with email. 5. Users enter email and password. 6. If user forgot password, click on forgot password option then password will recovered from registered email. 7. App validates credentials with Firebase authentication. 8. If valid, the user is redirected to the home screen.
Alternative Flows	If the entered email or password is incorrect, the user is shown an error message and remains on the login screen.
Business Rules	Email and password fields must be filled and validated. Email must be registered.

Assumptions	The user has a stable internet connection. Firebase services are available. The app is correctly configured to use firebase authentication.

Table 5:Use case 3

Use case 3

Use Case ID	UC-3
Use Case Name	Patient Health Management
Actors	Patient
Description	This use case describes all the actions a registered patient can perform within the Smart Healthcare Reminder application, including managing medications, recording health measurements, tracking daily activities, scheduling appointments, managing refills, updating intake status, accessing the AI chatbot, storing emergency contacts, and submitting feedback.
Trigger	The patient logs into the application and navigates to the patient dashboard.
Preconditions	Patient must be registered and logged into the system
Postconditions	All health data is stored, updated, and available for tracking and reminders/notifications are scheduled successfully

Normal Flow	1.Patient logs into the system 2.Access patient Dashboard 3. Enter disease information and send request to doctor and according to doctor's recommendation Manages medication, measurements, activities, appointments, and refill reminders 4. System schedules notifications for all reminders 5. Patient receives notifications 6. Patient marks tasks as Done or Dismiss 7. System updates health history and profile 8. Patient can view history, chatbot support and emergency contact anytime
Alternative Flows	Update/delete medication, activity, appointment, or measurement.
Business Rules	Missed tasks must be recorded in history Only valid numeric values allowed for health measurements
Assumptions	User has enabled notification permission. Internet is available if saving to firebase.

Table 6:Use case 4

Use case 4

Use Case ID	UC-4
Use Case Name	Caregiver
Actors	Caregiver

Description	The Caregiver module allows an authorized caregiver to monitor the patient's health activities including medication intake, measurements, daily routines and appointment status and ensure proper adherence to medical instructions.
Trigger	Caregiver logs into the application and selects a patient profile
Preconditions	Caregiver must be registered and linked with a patient account
Postconditions	Patient health activities are monitored and updated by caregiver
Normal Flow	1.Caregiver logs in 2. Accept or reject patient request 3. Views medication/activities/measurements 4. Updates task status 5. Monitors progress
Alternative Flows	Caregiver cannot access patient data if not authorized
Business Rules	Caregiver can only access assigned patient data Caregiver cannot modify medical prescriptions
Assumptions	User has enabled notification permission. Internet is available if saving to firebase.

Table 7:Use case 5

Use Case 5

Use Case ID	UC-5

Use Case Name	Pharmacist Medicine Delivery Management
Actors	Pharmacist
Description	This use case allows the pharmacist to manage patient medicine requests, provide medicine and delivery costs, review the patient's delivery address, and accept the delivery request.
Trigger	Patient sends a medicine request to the pharmacist.
Preconditions	Pharmacist is registered and logged into the Pharmacy Dashboard. Patient has submitted a medicine request.
Postconditions	Medicine and delivery costs are sent to the patient, and after the patient accepts the order and submits the delivery address, the pharmacist accepts the delivery request.
Normal Flow	1. Pharmacist logs into the Pharmacy Dashboard. 2. Pharmacists view the medicine request sent by the patient. 3. The pharmacist checks the medicine name and quantity. 4. Pharmacist enters the medicine cost and delivery cost and sends the cost details to the patient. 5. The patient accepts the medicine and delivery cost and submits the delivery address. 6. Pharmacist receives and reviews the delivery address and accepts the delivery request. 7. System updates the delivery request status.
Alternative Flows	If the medicine is unavailable, the pharmacist informs the patient.
Business Rules	Pharmacist must provide medicine and delivery costs before the patient can accept the order. Delivery address must be available before the pharmacist can accept the delivery request.
Assumptions	Pharmacist has access to the Pharmacy Dashboard. Firebase/Firestore is active and integrated. Patient and pharmacist have internet access.

Table 8:Use case 6

Use case 6

Use Case ID	UC-6
Use Case Name	Patient Health Monitoring & Recommendation
Actors	Doctor
Description	The Doctor module allows doctors to accept and review patient requests. Based on this data, doctors analyze patient conditions and provide recommendations about medicine, measurement, activities and caregiver support if required.
Trigger	Doctor logs into the system and selects a patient profile.
Preconditions	Doctor must be registered and authorized to access patient data
Postconditions	Patient receives medical recommendations and treatment plan
Normal Flow	1.Doctor logs in 2.Accept and review patient's request 3.Provides recommendations 4.View health record of patient 5.System updates patient profile
Alternative Flows	Doctor cannot access patient data without appointment linkage No data available
Business Rules	Only verified doctors can access patient data Doctor can only provide recommendations All recommendations must be recorded in system

Assumptions	Patient data is accurate and updated Doctor has internet access and system authorization

Table 9:Use case 7

Use Case 7

Use Case ID	UC-7
Use Case Name	In App Chat
Actors	Patient / Doctor/Caregiver
Description	Allow patient, doctor and caregiver to communicate directly through In App chat. User can discuss medical recommendations and health related issues.
Trigger	User open the chat section
Preconditions	Both Users are registered and logged in. Connection request has been approved between two users
Postconditions	Messages are successfully sent and saved in chat
Normal Flow	1. User open the chat section 2. User enters an sends a message 3. System Deliver the message 4. Receiver views and replies to the message
Alternative Flows	If the connection request is not approved the Chat option is not available.
Business Rules	Only approved users can access the chat
Assumptions	Users have internet access.

Table 10:Use case 8

Use Case 8

Use Case ID	UC-8
Use Case Name	Doctor Payment and Verification
Actors	Patient / Doctor
Description	This use case allows the patient to submit payment details to the doctor and allows the doctor to approve or reject the payment.
Trigger	Doctor provides Easypaisa or JazzCash payment details to the patient through chat after giving a recommendation.
Preconditions	Patient and doctor are registered and logged in. Doctor sent a recommendation. Doctor has provided Easypaisa or JazzCash payment details.
Postconditions	Payment is sent with Pending status and later updated to Approved or Rejected by the doctor.
Normal Flow	1. Doctor sent a recommendation. 2. Doctor sends Easypaisa or JazzCash number to the relevant patient through chat. 3. Patient opens Pay Doctor under the doctor section and enters doctor name and payment number. Patient enters transaction ID. Patient uploads payment receipt screenshot. 4. Patient sent payment details. 5. System saves payment with pending status. 6. Doctor opens the Payment section from the doctor dashboard. Doctor reviews the transaction ID and receipt. Doctor selects Approve or Reject. 7. System updates the payment status . Patient can view the updated payment status.
Alternative Flows	If payment information is incomplete, the system asks the patient to provide the missing information. If the doctor rejects the payment, the patient is shown Reject status.
Business Rules	Doctor payment details must be provided before payment. Transaction ID and receipt screenshot are required.

	Payment remains pending until checked by the doctor. Only the relevant doctor can approve or reject the submitted payment.
Assumptions	Payment information is submitted through the application. Patient and doctor have internet access.

Table 11:Use case 9

Use case 9

Use Case ID	UC-9
Use Case Name	Add medication
Actors	Patient/Caregiver
Description	This use case allows the user to add a new medicine to their medication list by entering details such as medicine name, dosage, frequency, date and timing.
Trigger	Users select the medication option from the home screen.
Preconditions	Users must be logged in. Medication details must be available to enter.
Postconditions	A new medication entry is entered into the database. Medication appears in the user medication list, and reminders are set
Normal Flow	1.Users open the app and navigate to the reminder section. 2. User taps on add medication 3. Users enter medicine's name, dosage, start date, end date, and reminder timings. 4. System validates the input. 5.System saves medication details to the database. 6. System set medication reminders.
Alternative Flows	System shows an error message and highlights missing fields.

Business Rules	Medication names and dosage fields are mandatory. Reminder time must be valid
Assumptions	User has enabled notification permission. Internet is available

Table 12:Use case 10

Use case 10

Use Case ID	UC-10
Use Case Name	Set Measurement Reminder
Actors	Patient/Caregiver
Description	Allows the user to set a reminder to check their health measurement (blood pressure, weight, or blood sugar).
Trigger	Users select the measurement option from the reminder section.
Preconditions	Users must be logged in.
Postconditions	A reminder is saved and set for notification.
Normal Flow	1. Users go to Measurement section. 2. Users select measurement type (BP, weight, sugar). 3. Users selects reminder time. 4. User set options(once a day, twice a day,3 times a day everyday, specific days) 5. System saves reminder. 6. System schedules notification for the selected time.
Alternative Flows	System shows users to select date and time if user does not enter it.

Business Rules	In measurement types date and time are mandatory.
Assumptions	User has enabled notification permission. Internet is available if saving to firebase.

Table 13:Use case 11

Use case 11

Use Case ID	UC-11
Use Case Name	Set Activity reminder
Actors	Patient/Caregiver
Description	This use case allows the user to set reminders for daily activities (walking, drinking water, eating).
Trigger	Users select activity module.
Preconditions	Users must be logged in.
Postconditions	A new activity reminder is created and scheduled for notification.
Normal Flow	1. Users open the activity section. 2. Users select the activity type(walk ,water ,meal or exercise). 3. Users select reminder type. 4. Users select Frequency (once a day,twice a day,3 times a day everyday, specific days) 5. System saves the activity reminder. 6. System set the notification.
Alternative Flows	If no activity type selected System shows warning message to select activity.
Business Rules	Each reminder must include a selected Activity type and time. Different reminders for the same activity type must have different time slots.

Assumptions	Users have stable internet connection.

Table 14:Use case 12

Use case 12

Use Case ID	UC-12
Use Case Name	Medical professional
Actors	Patient/Doctor/Caregiver
Description	Allows patient to send connection requests to doctors or caregivers, manage request status and schedule appointments.
Trigger	Patient opens the Medical Directory screen and selects the Doctor, Caregiver, or Appointments section
Preconditions	Users must be logged in.
Postconditions	Connection request is sent, request status is updated to pending and appointments are saved with reminder alerts.
Normal Flow	1.Patient open the medical professional screen. 2. Patient goes to doctor section enter details and symptoms and send request. 3. Users goes to caregiver section enter name and email and send request. 4. Patients open the appointment section to set medical visits with reminder.
Alternative Flows	If required fields are empty, the system displays an error message
Business Rules	Complete all required fields using a valid registered email to send requests.
Assumptions	Users have stable internet connection.

Table 15:Use case 13

Use case 13

Use Case ID	UC-13
Use Case Name	Set Appointment reminder
Actors	Patient/Caregiver
Description	Allows the user to schedule appointments by entering appointment details(date, time, doctor/clinic name). This use case allow user to set reminder for appointment.
Trigger	User selects add appointment
Preconditions	Users must be logged in.
Normal Flow	Users open the appointment module. Users select add appointments. Users enter the appointment date and time. Users enter a doctor name. System saves appointments. System set a reminder notification.
Alternative Flows	If date or time is missed, the system shows error and asks the user to fill required fields.
Business Rules	Appointments must have a valid date and time. Appointment reminder must come before the appointment.
Assumptions	Users have stable internet connection.

Table 16:Use case 14

Use case 14

Use Case ID	UC-14
Use Case Name	Set medicine refill reminder

Actors	Patient/Caregiver
Description	This use case allows the user to set refill reminder by entering refill details for a medicine. The System alerts the user when medicine stock is running low.
Trigger	User select the stock option form medication section. System finds remaining quantity has reached low- stock limit(e. 3 doses left)
Preconditions	Refill quantity checking must be enabled.
Postconditions	A refill reminder is set based on the medicine name or quantity.
Normal Flow	1. User open medication section. 2. Users enter medicine name. 3. Users enter quantity. 4. System validates the input. 5.User marks a dose as Done. 6.System updates the remaining quantity.
Alternative Flows	If medicine name or quantity is missed system shown user to enter required fields.
Business Rules	Refill Reminders will be sent when medicine will be low(e.g 3 units)
Assumptions	Users have stable internet connection

Table 17: Use Case15

Use Case 15

Use Case ID	UC-15
Use Case Name	Edit (Add, update, Delete) health tasks reminder time
Actors	Patient/Caregiver
Description	Allows to edit health task reminder time from the list.

Trigger	User select edit on a health task in the list
Preconditions	Users must be logged in. The health task reminder list must already exist.
Postconditions	Health task reminder time detail are updated in the database. Updated reminder times are rescheduled.
Normal Flow	1. User goes to the health task list screen. 2. User selects a module and click edit. 3. User updates fields (name, dosage, timing, dates) 4. System validates updated inputs. 5. System updates the database. 6. System set reminders according to new timings.
Alternative Flows	System displays validation errors. If the user makes no changes, the system keeps previous data.
Business Rules	Edited data must follow the same constraint as adding health task reminder time.
Assumptions	Internet is available

Table 18:Use Case 16

Use case 16

Use Case ID	UC-16
Use Case Name	Measurement tracker
Actors	Patient/Caregiver
Description	Allow users to record and track their health measurement such as blood pressure, sugar level and body weight.
Trigger	User select the tracker icon.
Preconditions	User must be logged in. Measurement type must be selected.

Postconditions	The entered measurement data is saved in the database. The record becomes available in the history in tracker.
Normal Flow	1. User navigates to the Measurement Tracker section. 2. User selects measurement type (BP, weight, sugar level). 3. User enters the measurement values (e.g., systolic/diastolic for blood pressure, sugar level , or weight). 4. System validates the entered data. 5. System saves the measurement record in tracker section.
Alternative Flows	System displays message to user if measurement values are missing. System displays validation error if entered values are invalid.
Business Rules	Measurement values must be entered in valid numeric format.
Assumptions	Internet is available

Table 19 :Use Case 17

Use Case 17

Use Case ID	UC-17
Use Case Name	Pharmacy Medicine Delivery
Actors	Patient/Caregiver
Description	This use case allows the patient to enter medicine name, quantity, pharmacy name, pharmacy address and phone number and send request to pharmacist which confirm medicine is in stock and confirm or reject delivery.
Trigger	The patient opens the Pharmacy section and selects the option to order medicine.
Preconditions	The patient must be logged into the application. Pharmacy and medicine stock information must be available.

Postconditions	If the medicine is available, the order is successfully placed and the order details are saved. If the medicine is unavailable, an error message is displayed.
Normal Flow	1. The patient opens the Pharmacy screen. 2. The patient enters the medicine name and required quantity. 3. The patient enters the pharmacy name and address and pharmacy manager email.
Alternative Flows	If the requested medicine is out of stock or delivery is unavailable, the system displays an error message and the patient cannot place the order
Business Rules	Medicine name and quantity must be entered. Delivery address must be provided. The requested medicine must be available in stock.
Assumptions	The patient has a stable internet connection. Pharmacy stock and delivery information are available. The patient provides a valid delivery address.

Table 20:Use Case 18

Use case 18

Use Case ID	UC-18
Use Case Name	Chatbot Assistance
Actors	Patient/Caregiver
Description	This use case allows the user to interact with the chatbot to get guidance on general health tips.
Trigger	User taps the Chatbot icon from the dashboard
Preconditions	User must be logged in.

	Chatbot service must be active.
Postconditions	User receives guidance from the chatbot
Normal Flow	1.User opens the chatbot from the application. 2.User types a question. 3.System processes the request using chatbot logic. 4. Chatbot provides answers, health tips or guidance.
Alternative Flows	If the question is unclear the chatbot asks for clarification.
Business Rules	Chatbot responses must be general health guidance only and not provide professional medical advice. Chatbot should respond within a few seconds to maintain user connection.
Assumptions	User has enabled internet access for chatbot interaction.

Table 21:Use Case 19

Use case 19

Use case Id	UC-19
Use Case Name	Receive notification for reminders and messages
Actors	Patient/Caregiver
Description	System send notification to remind user about Medication , Measurements, Activities, Appointments, refills and messages between users.
Trigger	The reminder time is reached.
Preconditions	Notification permission must be enabled.
Postconditions	Users receive notification at the correct time.

Normal Flow	1. User sets a reminder (medicine, measurement, Activity, Appointment, refill) and Sends a message 2. System set notifications. 3. When time arrives, system sends a notification. 4. Users view the notification.
Alternative Flows	If notification permission is off Systems remind user to enable notification.
Business Rules	Notification must be sent at the reminder time. Each reminder must give only one notification when user marked it as Done or Dismiss. Notification will clearly tell the type of reminder.
Assumptions	Internet connection is available when notification are sent.

Table 22:Use Case 20

Use case 20

Use Case ID	UC-20
Use Case Name	Track intake
Actors	Patient/Caregiver
Description	This use case allow user to update the intake status of reminders Done or Dismiss to maintain daily health progress record. If patient Dismiss the task notification will be show to caregiver that relevant patient Dismiss its task.
Trigger	User receive a reminder notification and select taken or missed.
Preconditions	User must be logged into the application. Reminder time selected for the current day.
Postconditions	Intake status is stored for daily progress tracking. Reminder history is updated in the database.

Normal Flow	1.The system sends a scheduled reminder notification. 2. The user selects an intake status(taken, missed). 3. The system saves the selected status in firebase. 4. Through daily progress history is updated.
Alternative Flows	If the user ignores the reminder, the system will not mark the status until the user marks it
Business Rules	Intake status must be recorded within the same day.
Assumptions	. Internet is available

Table 23 :Use Case 21

Use case 21

Use case Id	UC-21
Use Case Name	View profile
Actors	Patient/Caregiver
Description	Allows users to view their health related summary in history section located in profile
Trigger	User taps the profile icon from the menu.
Preconditions	Users must be logged in. Profile data must exist in the database.
Postconditions	Profile details are displayed on screen.
Normal Flow	User opens the app and selects profile then go to history section where user health reminders are shown which are marked as taken or missed.
Alternative Flows	If the user is not logged in the system redirects to the login/signup screen.
Business Rules	Only authenticated user can access their profile.

Assumptions	User has enabled internet connection.

Table 24:Use Case 22

Use case 22

Use case Id	UC-22
Use Case Name	Admin
Actors	Admin
Description	This use case allows admin to view users and approval or rejection of doctor, pharmacist and nurse license and feedback to ensure system work smoothly.
Trigger	Admin logs into the system and access the admin dashboard.
Preconditions	Admin must be logged in
Postconditions	User data is managed successfully. Feedback is viewed.
Normal Flow	1.Admin logs into the system. 2.Admin opens the dashboard. 3.Admin views user accounts. 4.Admin approval or rejection of doctor, pharmacist and nurse license. 5.Admin open feedback section. 6.Admin views user feedback.
Alternative Flows	If login fails, the system asks the admin to enter valid credentials. If no users are available, the system displays an empty list.
Business Rules	Only authorized admin can access admin functionalities.
Assumptions	Admin has proper access rights. The user has enabled internet connection.

Table 25:Use Case 23

Use case 23

Use case Id	UC-23
Use Case Name	Feedback
Actors	Patient/Caregiver/Doctor, Admin
Description	This use case allow users to send their suggestions, complaints or questions to the system then admin can view these messages to help users and improve the system.
Trigger	User select the Feedback option in profile section.
Preconditions	User must be logged in.
Postconditions	Feedback is stored in the system. Admin can view feedback.
Normal Flow	1. User open feedback section in profile icon. 2. User submits feedback. 3. System stores feedback in firebase. 4. Admin views feedback.
Alternative Flows	System ask user to enter feedback if field is empty.
Business Rules	Feedback must include valid message.
Assumptions	Internet connection is available.

Table 26:Use Case 24

Use case 24

Use case Id	UC-24
Use Case Name	Emergency contact
Actors	Patient/Caregiver

Description	This use case allows the user to enter emergency contact details such as phone number so it can be quickly accessed in case of an emergency.
Trigger	Users select the Emergency contact option in the profile section.
Preconditions	Users must be logged in.
Postconditions	Emergency contact is saved or it can be updated. Users can quickly access.
Normal Flow	1. Users open the emergency contact section. 2. User selects Add Contact details(name, phone number). 3. The system validates the input. 4. The system saves the contact in the database.
Alternative Flows	The system requests the user to enter valid details if required fields are empty. The system asks the user to add a contact if no contact exists.
Business Rules	Contact must have a valid phone number.
Assumptions	Firebase is available for data storage.

2.5 Functional Requirements

This section describes the functional requirements of the system expressed in natural language style. This section is organized by features as a system feature name and specific functional requirements associated with this feature. So for each module and its features the functional requirements are itemized below:

Table 27:Functional Requirement 1

Functional Requirement 1

Identifier	FR-1

Title	User profile management
Requirement	The user shall be able to create and manage their profile by entering personal information such as full name, Email, Date of birth, gender and password. When a user select their role it will also enter a license number or speciality.The user shall be able to login by entering Email and password then enter login.
Source	User Requirement, System Requirement
Rationale	It provides a secure environment and collects profile data.
Business Rule	User Date of Birth must follow a format(YYYY/MM/DD),Gender selection must be from predefined options(Male, Female, others)
Dependencies	Internet connection, Firebase authentication
Priority	Medium

Table 28:Functional Requirement 2

Functional Requirement 2

Identifier	FR-2
Title	Forgot Password
Requirement	The end user will be able to recover the password by entering a registered email and receiving a password.
Source	System Requirement

Rationale	Support work will be reduce and user will recover their password easily.
Business Rule	Password should expire after a limited time.
Dependencies	FR-1
Priority	Medium

Table 29:Functional Requirement 3

Functional Requirement 3

Identifier	FR-3
Title	Patient Health Management
Requirement	The system allows patients to manage medications, health data, daily activities, and appointments. It supports intake updates, reminders, pharmacy, chatbot access, history, emergency contacts and feedback.
Source	User Requirement
Rationale	This feature helps patients manage their health efficiently in one place, reduces missed medications, and improves overall care and monitoring
Business Rule	Record tasks, validate numeric data and send reminders
Dependencies	Authentication, database, notifications and internet connection
Priority	High

Table 30:Functional Requirement 4

Functional Requirement 4

Identifier	FR-4
Title	Caregiver Assistance
Requirement	The system allows caregivers to monitor a patient health activities and update task status. They can monitor progress and help the patient.
Source	User Requirement
Rationale	This feature ensures proper patient care and support by enabling caregivers to help patients in managing their health effectively.
Business Rule	Caregiver can only access their assigned patients
Dependencies	Internet connection, Firebase authentication
Priority	High

Table 31:Functional Requirement 5

Functional Requirement 5

Identifier	FR-5
Title	Pharmacist Medicine and Delivery Management
Requirement	The system shall allow the pharmacist to review patient medicine requests, provide the medicine cost and delivery cost, and send the cost details to the patient. After the patient accepts the cost and submits a delivery address, the pharmacist review and accept the delivery request.

Source	User Requirement, System Requirement
Rationale	It provides an combined process for managing medicine requests, cost confirmation, and delivery between patients and pharmacists.
Business Rule	Pharmacist must provide medicine and delivery cost.The pharmacist must review and accept the delivery request before proceeding with delivery.
Dependencies	Internet connection, Firebase/Firestore, Patient Pharmacy Module
Priority	High

Table 32:Functional Requirement 6

Functional Requirement 6

Identifier	FR-6
Title	Doctor
Requirement	The system shall allow authorized doctors to access patient health records, analyze data and provide medical recommendations and treatment plans.
Source	User Requirement, System Requirement.
Rationale	This feature enables doctors to make informed decisions and provide accurate treatment based on patient data.
Business Rule	Only authorized doctors can access data, cannot modify patient-entered records, and must record all recommendations.

Dependencies	Internet connection, Firebase authentication
Priority	High

Table 33:Functional Requirement 7

Functional Requirement 7

Identifier	FR-7
Title	In-app chat
Requirement	The system shall allow the doctor and patient to communicate through an in-app chat. The chat option shall become available after the doctor approves the patient's request .The chat option will become available when caregiver approves the request. The Doctor, patient and caregiver can all communicate through chat when doctor gives recommendation by both patient and caregiver they can all do group chat and private chat individually.
Source	User Requirement, System Requirement
Rationale	It provides a convenient way for users to communicate regarding recommendations and healthcare-related concerns.
Business Rule	Only an approved relationship can access the chat. The doctor, patient and caregiver can send and receive messages through their respective dashboards.
Dependencies	Internet connection, Firebase/Firestore
Priority	High

Table 34:Functional Requirement 8

Functional Requirement 8

Identifier	FR-8
Title	Doctor's payment
Requirement	The system shall allow the doctor to send Easypaisa or JazzCash payment details to the relevant patient through chat after submitting a recommendation. The patient shall be able to enter the doctor name, payment number, transaction ID, and upload a payment receipt screenshot.
Source	User Requirement, System Requirement
Rationale	It provides a convenient method for patients to submit payment information to the doctor and allows the payment to be verified through the application.
Business Rule	Transaction ID and payment receipt are required for payment submission. Payment status shall remain Pending until the doctor reviews it. The doctor can Approve or Reject the payment.
Dependencies	Internet connection, Firebase/Firestore, Doctor–Patient Chat
Priority	High

Table 35:Functional Requirement 9

Functional Requirement 9

Identifier	FR-9
Title	Add medication
Requirement	The end user will be able to add medication to their treatment plan by entering the medicine name, dosage, frequency, start date, end date, and reminder time.

Source	User requirement
Rationale	Users can properly manage their prescribed medicines and take them on time.
Business Rule	Medicine name cannot be empty, start date cannot be later than end date, frequency must be selected from predefined options (e.g. once daily, twice daily), Reminder time must be a valid time format.
Dependencies	Notification manager, User profile, Firebase Fire store
Priority	High

Table 36:Functional Requirement 10

Functional Requirement 10

Identifier	FR-10
Title	Set measurement reminder
Requirement	The user shall be able to set reminders for health measurement such as blood pressure, blood sugar and body weight by selecting the measurement type and preferred reminder time.
Source	User Requirement
Rationale	Regular health measurement help user monitor their health conditions effectively and detect health issues at early stage.
Business Rule	The user select measurement type and valid reminder time.
Dependencies	FR-1,FR-11,FR-12
Priority	High

Table 37:Functional Requirement 11

Functional Requirement 11

Identifier	FR-11
Title	Set Activity reminder
Requirement	The user shall be able to set a daily activity reminder (such as walking, drinking water, or eating meals,Yoga or cycling) by selecting an activity type and time.
Source	User Requirement.
Rationale	Supports healthcare by promoting regular walking, hydration, and proper eating.
Business Rule	User can create multiple activity reminders (e.g. morning walk + water reminder)
Dependencies	FR-1,FR-11,FR-12
Priority	Medium

Table 38:Functional Requirement 12

Functional Requirement 12

Identifier	FR-12
Title	Medical Professional
Requirement	The system shall allow patients to send connection requests to doctor and caregiver and schedule appointments with reminder
Source	User Requirement, System Requirement

Rationale	Enables direct connection between patients, doctors, and caregivers
Business Rule	Mandatory fields must be filled, recipient emails must be valid and registered, and appointments must be set for future dates.
Dependencies	Internet connection, Firebase/Firestore
Priority	High

Table 39:Functional Requirement 13

Functional Requirement 13

Identifier	FR-13
Title	Appointment Reminder
Requirement	The user shall be able to schedule doctor's appointments by entering the date, time, and doctor name.
Source	User requirement
Rationale	Helps users avoid missing doctor visits and useful for chronic ill patients who have regular checkups
Business Rule	Appointment reminders must notify the user at least once .Date and time must be valid and cannot be empty.
Dependencies	FR-1,FR-21
Priority	High

Table 40:Functional Requirement 14

Functional Requirement 14

Identifier	FR-14
Title	Medicine Refill Reminder
Requirement	The system shall inform the user when medicines are about to finish based on the medicine name or stock entered by the user in medication section.
Source	System requirement, User requirement.
Rationale	Ensures user never miss important medicine.
Business Rule	Medicine stock will be less than 3 notification will be shown with saved medicine.
Dependencies	FR-1,FR-9
Priority	High

Table 41:Functional Requirement 15

Functional Requirement 15

Identifier	FR-15
Title	Edit (Add, update, Delete) health task
Requirement	The user will be able to edit health task reminder.
Source	User requirement.

Rationale	Prevents incorrect reminders for health task. Users often change their dosage, timing or discontinue task.
Business Rule	Edited data will follow the same constraint as adding health task reminder time .
Dependencies	FR-1, FR-9,FR-10,FR-11,FR-13
Priority	High

Table 42:Functional Requirement 16

Functional Requirement 16

Identifier	FR-16
Title	Record measurement data (Measurement Tracker)
Requirement	The user shall be able to record and store health measurement data such as blood pressure(systolic/diastolic), blood sugar level and body weight for tracking and monitoring their health over time.
Source	Business Requirement, interview
Rationale	Recording health measurement helps users monitor their health condition and observes changes over time.
Business Rule	The user must enter valid numeric values for measurements (e.g. systolic/diastolic for BP, sugar level or weight).
Dependencies	FR-1, FR-10
Priority	High

Table 43:Functional Requirement 17

Functional Requirement 17

Identifier	FR-17
Title	Pharmacy Medicine Delivery

Requirement	The system shall allow patients to send medicine requests to a pharmacist by entering the medicine name, quantity, pharmacy name, pharmacy address, and phone number. The pharmacist shall receive the request through the Pharmacy Dashboard.
Source	Business Requirements, user interviews
Rationale	It allows patients to conveniently request required medicines from a selected pharmacy through the application.
Business Rule	Medicine name and quantity must be provided by the patient. Pharmacy information must be provided before the request can be submitted.
Dependencies	FR-1, FR-5
Priority	Medium

Table 44:Functional Requirement 18

Functional Requirement 18

Identifier	FR-18
Title	Chatbot Assistance
Requirement	The user shall be able to interact with chatbot to ask health related questions and guidance about reminders .
Source	Business Requirements, user interviews
Rationale	Helps users quickly get health guidance and information without searching manually. It will make the application more interactive and user-friendly.
Business Rule	The chatbot must respond only to predefined health-related queries and should provide simple and clear responses.
Dependencies	FR-1, FR-3,FR-4
Priority	Medium

Table 45:Functional Requirement 19

Functional Requirement 19

Identifier	FR-19
Title	Receive Notification for reminder and chat messages
Requirement	System send notification to remind user about health task and messages.
Source	System Requirements.
Rationale	Help user to remind their task and messages
Business Rule	Notification must be sent at the reminder time and when message is send it will be received at that time.
Dependencies	FR-1, FR-9,FR-10,FR-11,FR-13
Priority	Medium

Table 46:Functional Requirement 20

Functional Requirement 20

Identifier	FR-20
Title	Track intake
Requirement	The user shall be able to mark routine as “Done" or "Dismiss” and mark as patient or caregiver should also mark and the system shall update daily progress accordingly. If patient mark as dismiss the notification will be sent to caregiver that relevant patient Dismiss its task.
Source	System Requirement ,User Requirement.
Rationale	To help users maintain regular routine.
Business Rule	Once marked, the intake status is saved with date and time for progress tracking. Intake status can only be marked when notification will send.

Dependencies	FR-1, FR-9,FR-10,FR-11,FR-13
Priority	High

Table 47:Functional Requirement 21

Functional Requirement 21

Identifier	FR-21
Title	View user profile
Requirement	The user shall be able to view personal information and a summary of health reminders and intake progress.
Source	user Requirement
Rationale	Viewing the user profile helps user review and manage their personal details and monitor their overall health reminder progress in one place.
Business Rule	Only authenticated user can access their own profile. Health progress shown must be calculated from recorded intake data.
Dependencies	FR-1,FR-22
Priority	High

Table 48:Functional Requirement 22

Functional Requirement 22

Identifier	FR-22
Title	Admin

Requirement	The system shall allow admin to manage users and approval or rejection of doctor, pharmacist and nurse license and handle user feedback.
Source	System requirement
Rationale	This feature ensures proper system control, monitoring and user support by allowing admin to manage core functionalities.
Business Rule	Admin can view and manage user data. Admin can accept or reject user's license. Admin can view feedback.
Dependencies	FR-1,FR-13,FR-17
Priority	high

Table 49:Functional Requirement 23

Functional Requirement 23

Identifier	FR-23
Title	Feedback
Requirement	The system will allow to submit their complaints or queries about the system and enable admin to view it.
Source	User requirement
Rationale	This feature helps improve the system by collecting user opinions.
Business Rule	Only authenticated users can submit feedback. Admin can view feedback.
Dependencies	FR-1,FR-3,FR-4,FR-6

Priority	High

Table 50:Functional Requirement 24

Functional Requirement 24

Identifier	FR-24
Title	Emergency Contact
Requirement	The system allow user to add, update and access emergency contact.
Source	User Requirement
Rationale	This feature enhances user safety by allowing quick access to important contact such as Doctor's PA number or name.
Business Rule	Only authenticated users can manage their emergency contact. Emergency contact must have a valid phone number.
Dependencies	FR-1
Priority	High

2.6 Non-Functional Requirements

Non-functional Requirements are the constraints that specifies the quality attribute of software. They are important because they can significantly impact the overall quality and success of software system.

The following are non-functional requirements:

2.6.1 Reliability

The Smart Healthcare Reminder application should work smoothly and available most of the time. The application should recover automatically if small errors occur and keep important data safe like reminders, intake tracking and user profile.

2.6.2 Usability

The application should be easy to use for all users including patients, caregiver, doctor, pharmacist and administrator. The system should have a simple and clear interface so users should easily perform tasks like adding reminders, track intake, viewing profiles and using the chatbot. The application should have a clear button, readable text and simple navigation so new users can understand it quickly. The system should show a clear message to guide the user if any error occurs.

2.6.3 Performance

The application should load screens quickly under normal conditions. It should handle multiple users at the same time without slowing down. Features like saving reminders, recording measurement values, and chatbot respond smoothly within minimal delay.

2.6.4 Security

The application should protect all user data such as personal information, health record, reminders. Only authorized users can access the system and each user can access their own data. The system should use role based access so that admin and users have different accessibility.

2.7 External interface Analysis:

2.7.1User Interface Requirements

- The app follows standard design principles for clarity and ease of use.
- It will adjust to different screen sizes and devices.
- Users will receive clear notifications for their actions.

2.7.2 Software Interfaces

- The app will run on android.
- Firebase will be used for data storage and management.

2.7.3 Hardware Interfaces

- Users will access the application mainly through smartphones and tablets.

2.7.4 Communication Interfaces

- The application will use standard mobile communication methods and secure protocols for data exchange.

2.7.5 Error handling and Reporting

- The smart healthcare reminder application handles errors by checking user input, login details, internet connection and notification permission. When a problem occurs, clear messages are shown to help users to fix it, which keeps the application reliable and easy to use.

2.8 Summary

Chapter 2 outlines the requirements of Smart Healthcare Reminder application detailing how user can register, set reminders for medication, measurement, activity, appointment and refills and track their daily intake. This chapter also outlines non-functional requirements such as reliability, usability, security along with system interfaces used for notification, data storage and user interaction.

Chapter 3

Design and Architecture

3.System Design

This chapter explains the overall system design and its detailed analysis. It includes key design consideration, System Structure. The Design models used in this project include sequence diagrams, state diagrams, class diagrams, and component diagrams, each representing a different perspective of the system's behavior and structure. For the application architecture, the MVVM (Model-View-View Model) pattern is adopted because it provides clear separation of responsibilities, improves modularity and enhances testability and maintainability of code.

3.1 Design Considerations

Assumptions and Dependencies:

Following assumptions and dependencies were considered during the system design, the user will have stable internet connection for data synchronization and authentication through Firebase services. The user is also expected to use an android device running version 7 or above with google play service enabled, as notification and background reminder functionality depends on it. The system assumes that the device has at least 4GB of RAM and 3GB of free storage to install the application.

Most assumptions, dependencies, and system constraints are further discussed in chapter 1.

Limitations

1. The application is designed for mobile use and may have limited functionality on desktop browsers.
2. The application does not provide medical diagnosis.

Risks:

1. There is a risk of downtime if the firebase server experiences issues, which could disrupt access user the app.
2. Sensitive user data could be at risk if security measures like authentication are not properly implemented.

3.2 Design Models

Design models provide structured representations of the system from multiple viewpoints. It is important to note that a design model does not attempt to fully replicate the entire system. A representation tries to capture every detail of an entity, whereas an abstraction focuses on simplifying the system to bring

forward its most significant features. Such models are used to convey the view of application to other stakeholders. It purposely leaves out detail to make it easier to understand.

1:Class Diagram

Figure 2:Class Diagram

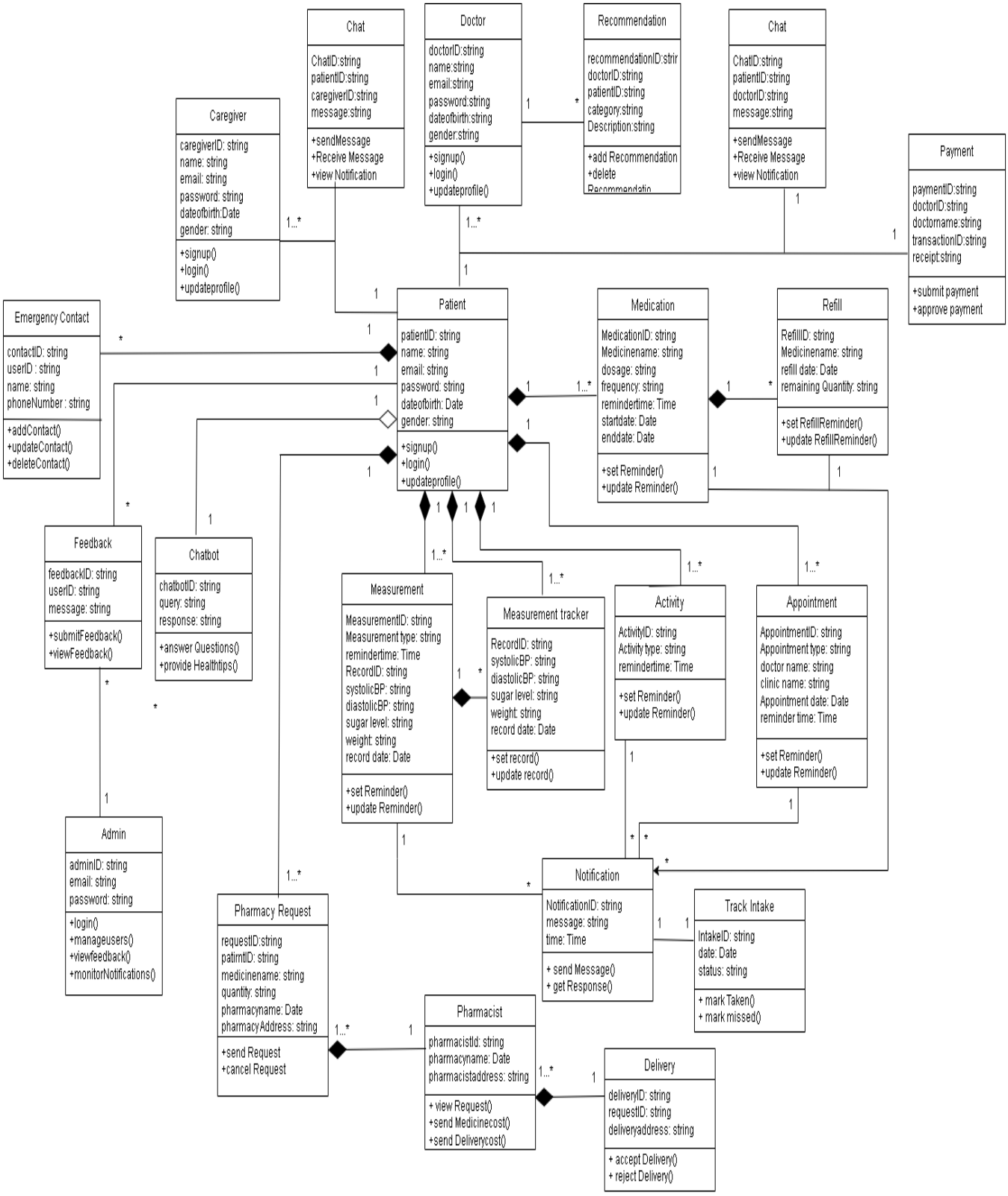


Figure 3.1 Class Diagram

2: Sequence Diagram

Figure 3:Sequence Diagram

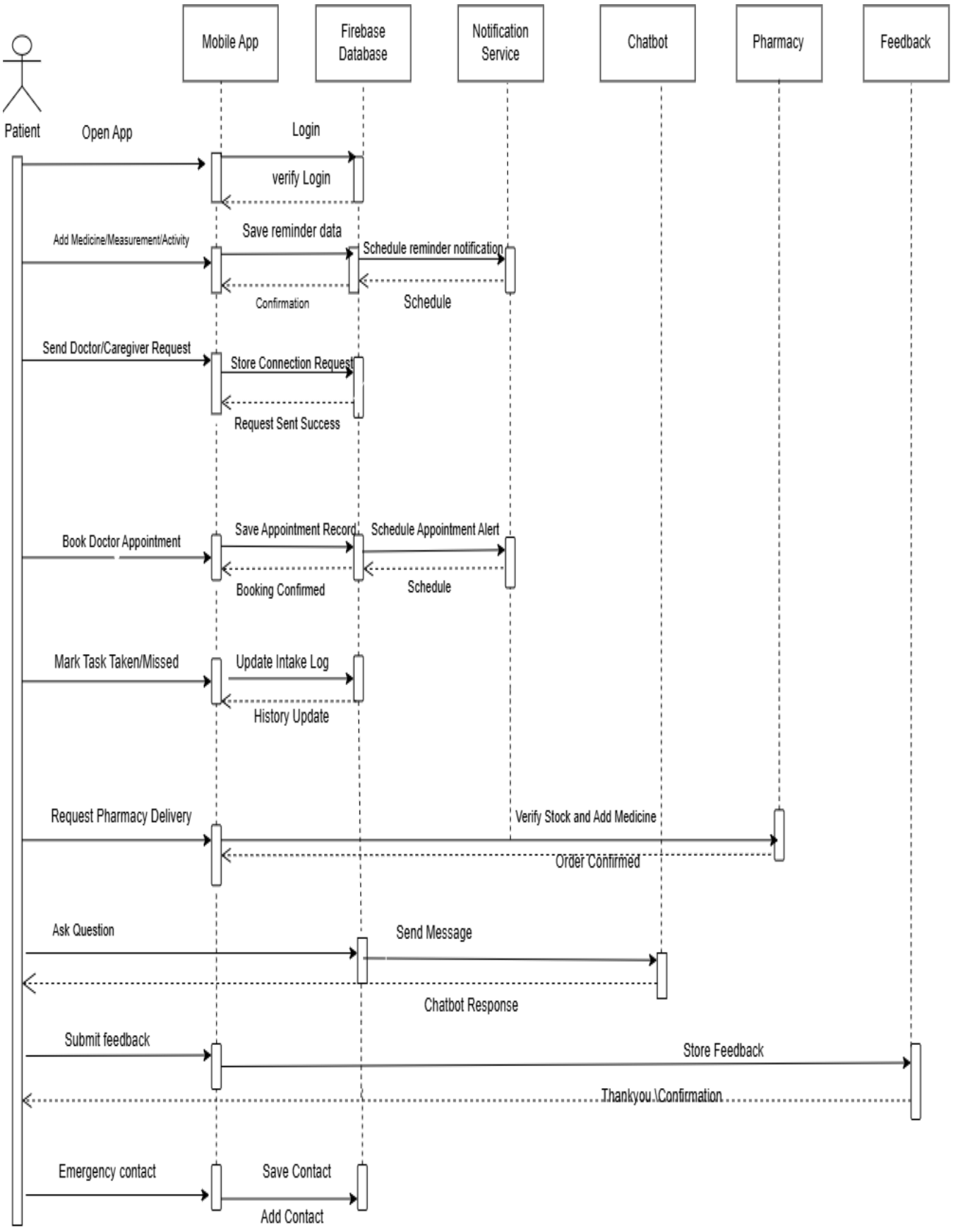


Figure 3.2 Sequence Diagram for Patient

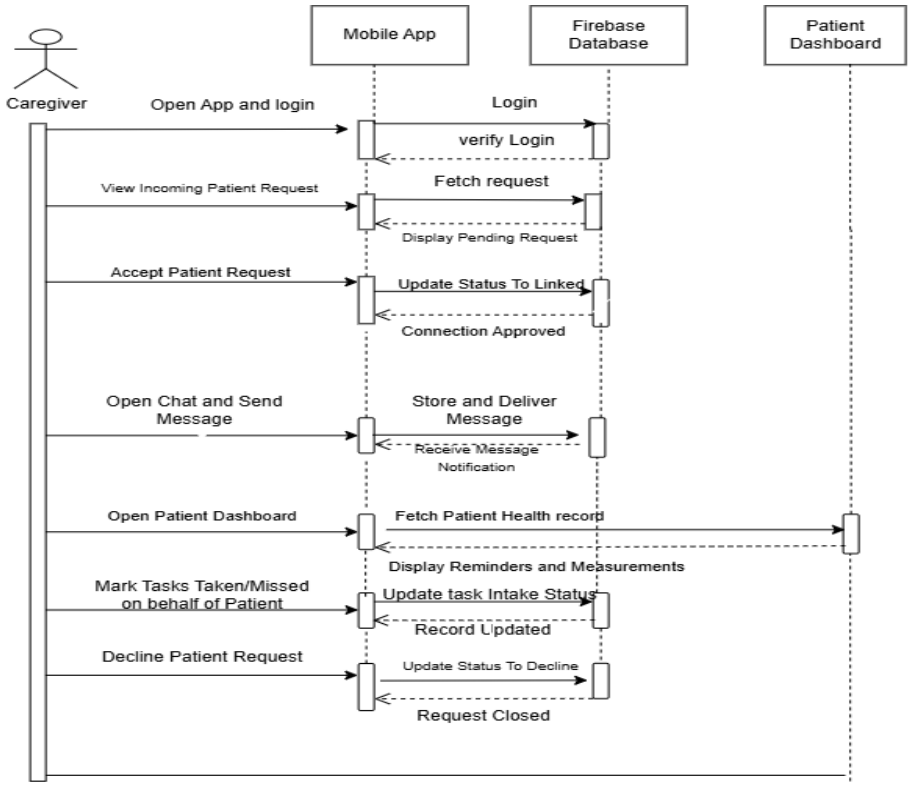


Figure 3.3 Sequence Diagram for caregiver

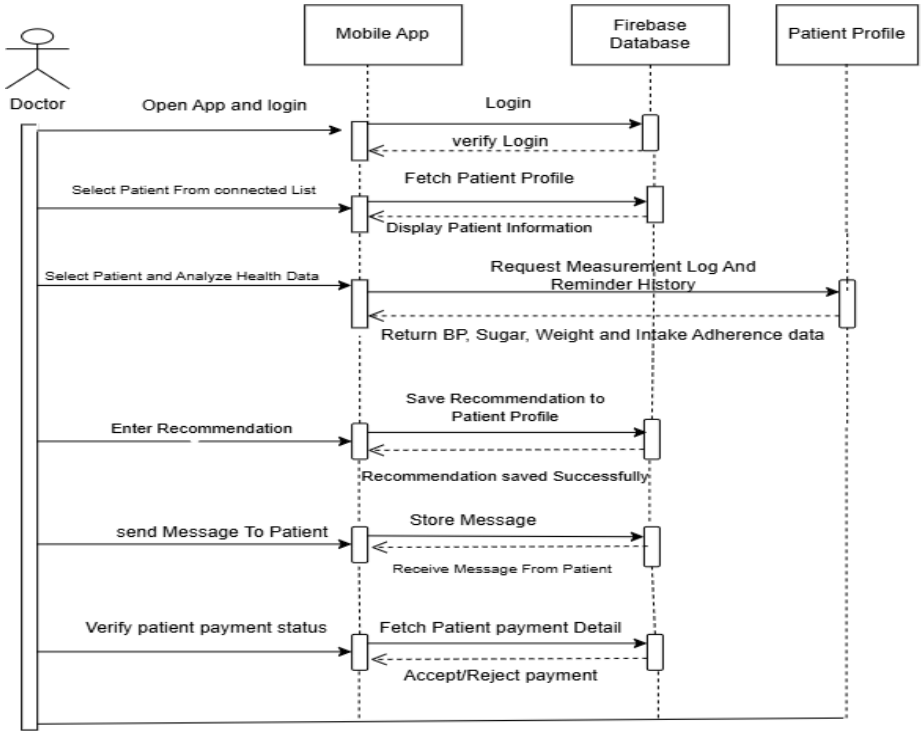


Figure 3.3 Sequence Diagram for Doctor

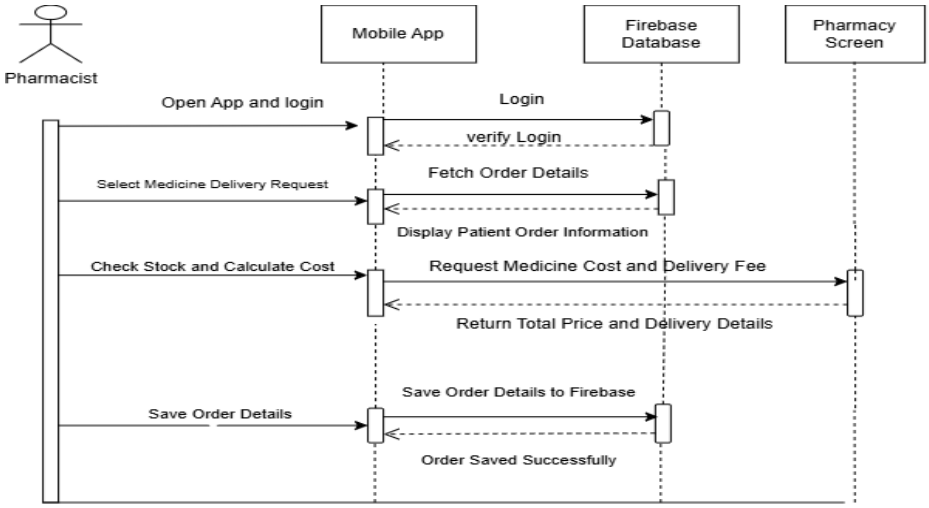


Figure 3.4 Sequence Diagram for Pharmacist

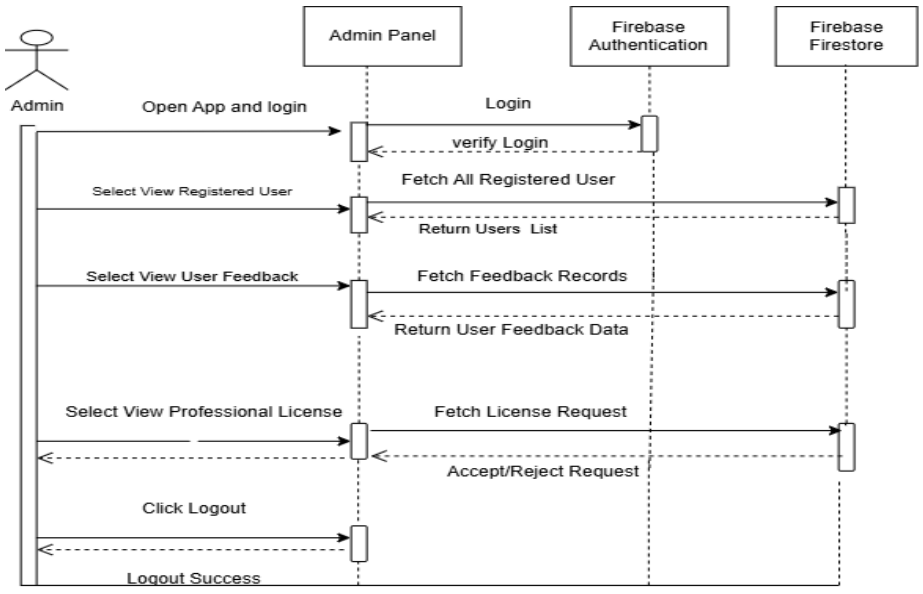


Figure 3.5 Sequence Diagram for Admin

3:State Diagram

Figure 4:State Diagram

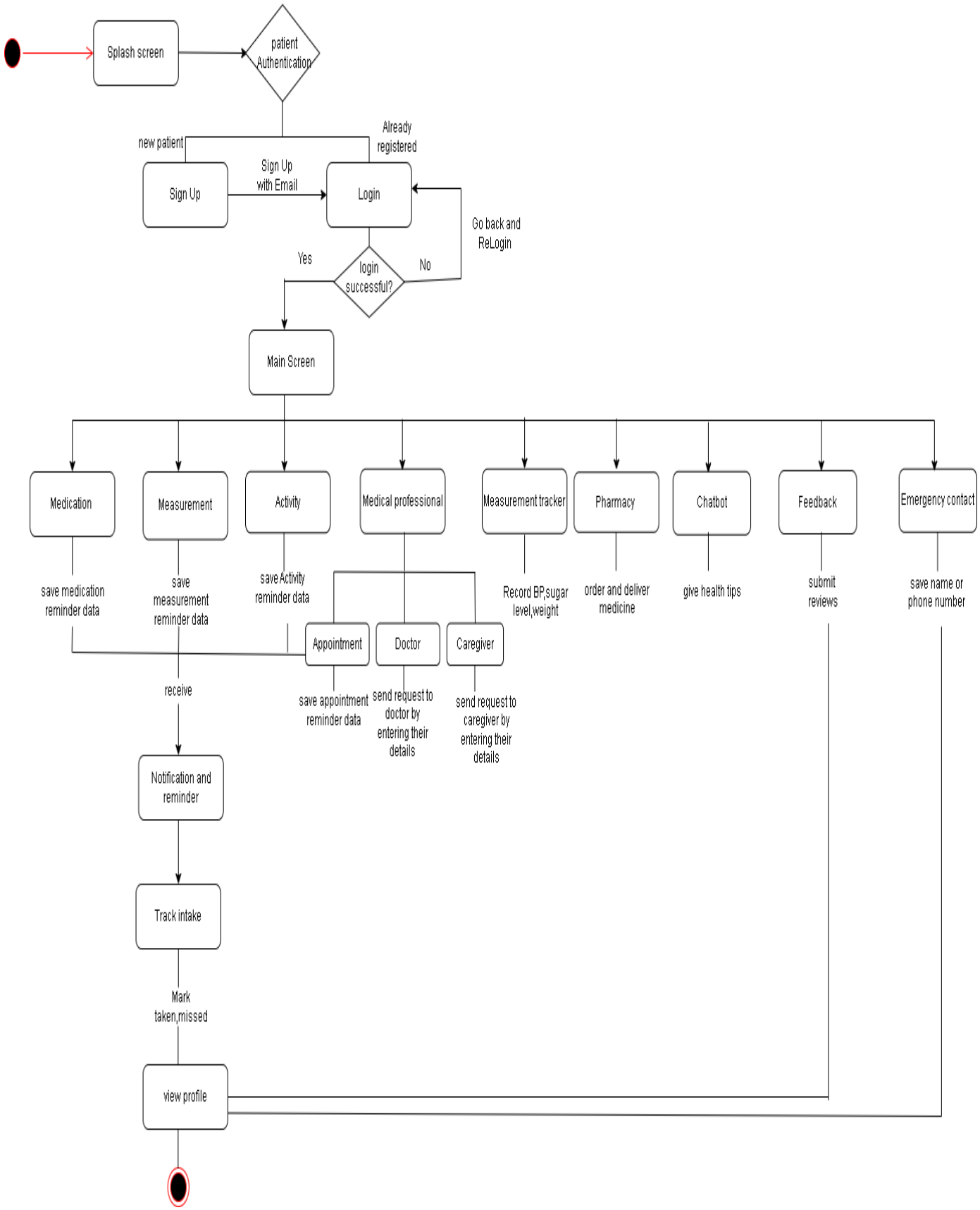


Figure 3.4 State Diagram for patient

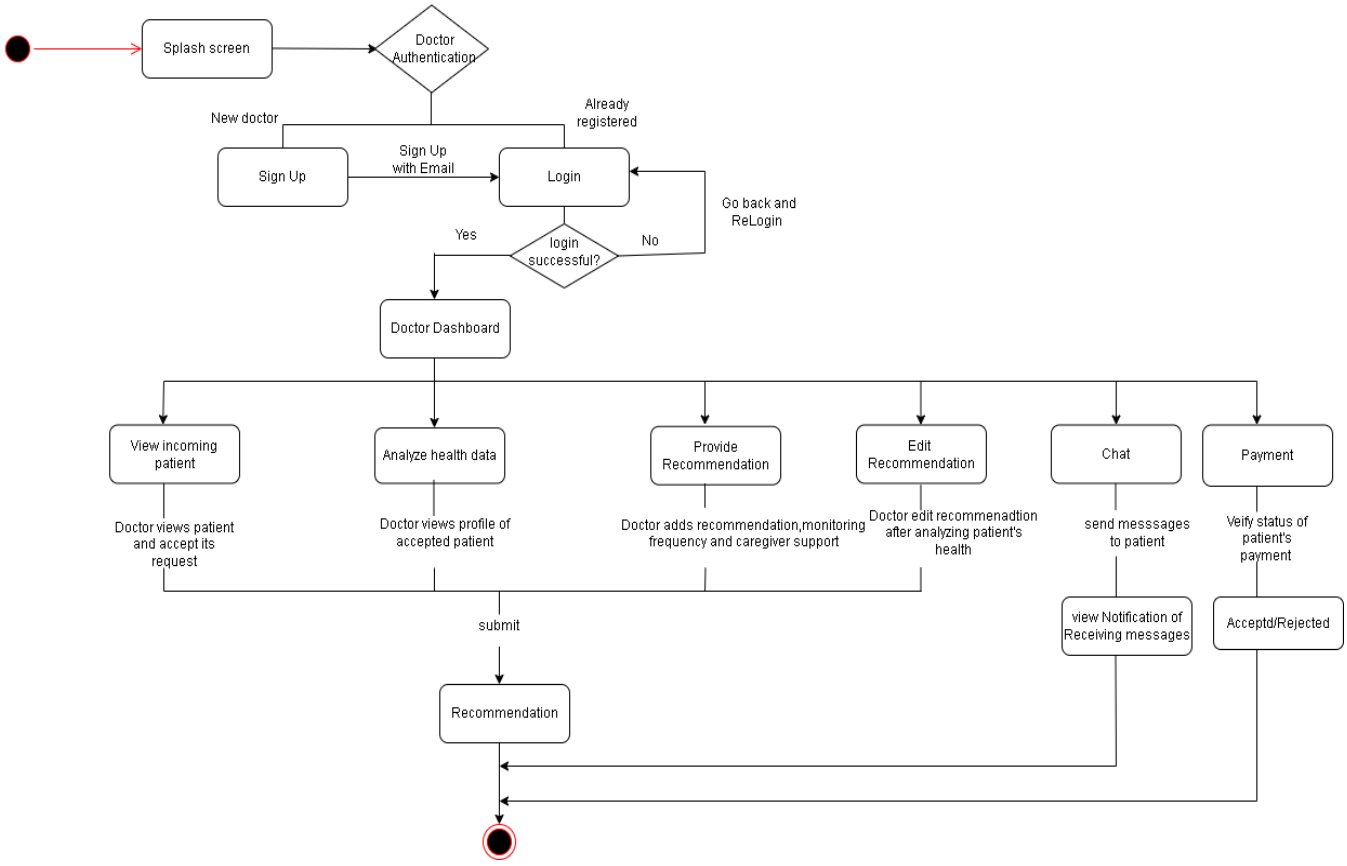


Figure 3.4 State Diagram for doctor

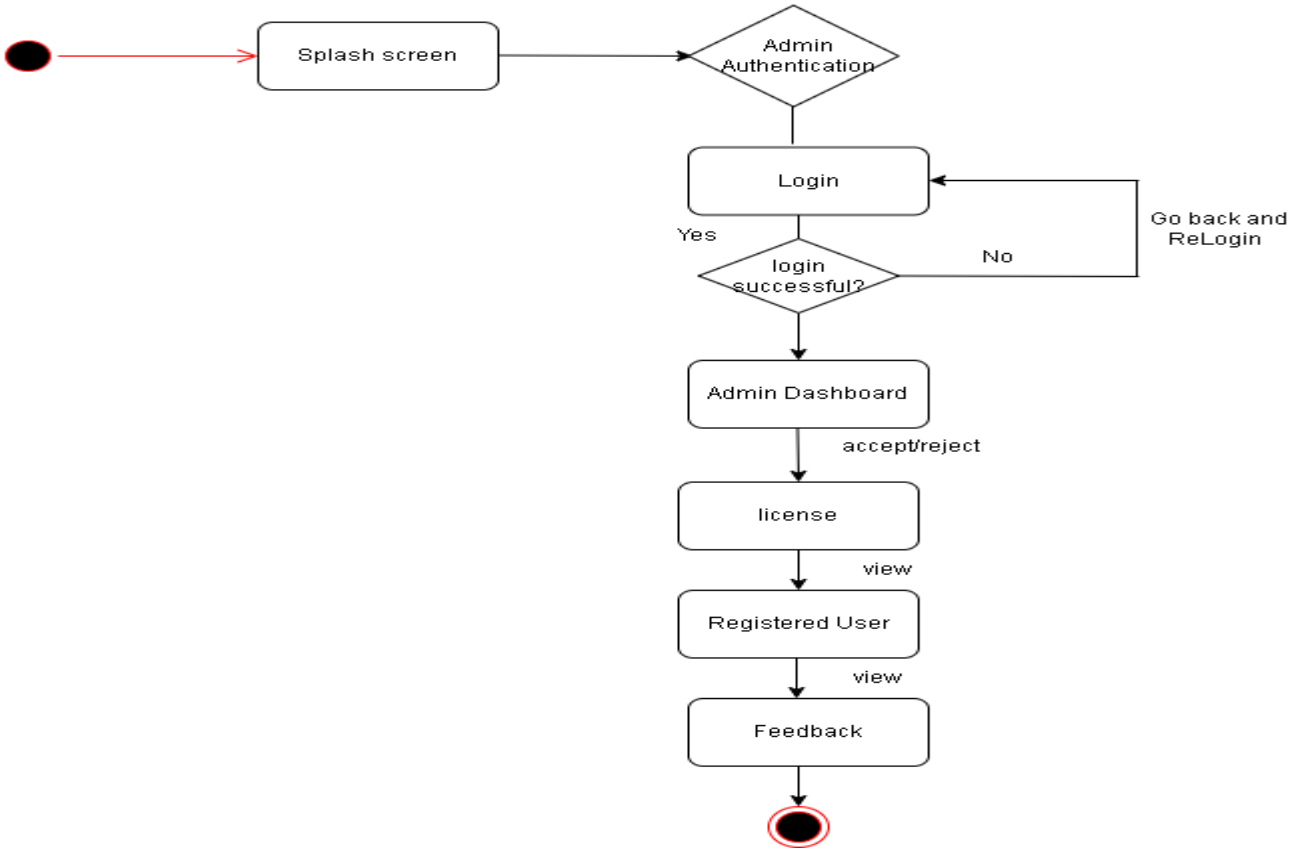


Figure 3.5 State Diagram for Admin

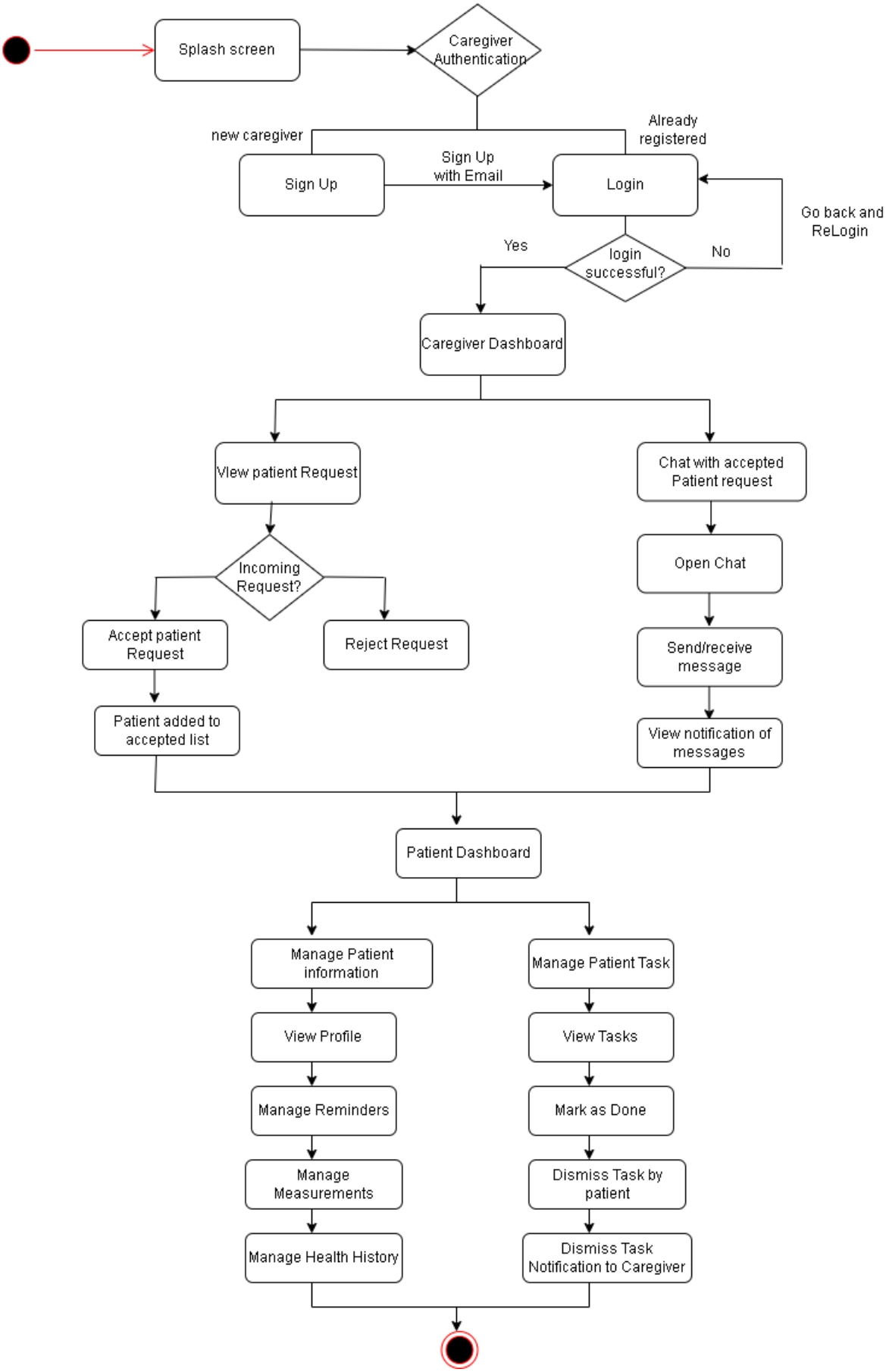


Figure 3.4 State Diagram for caregiver

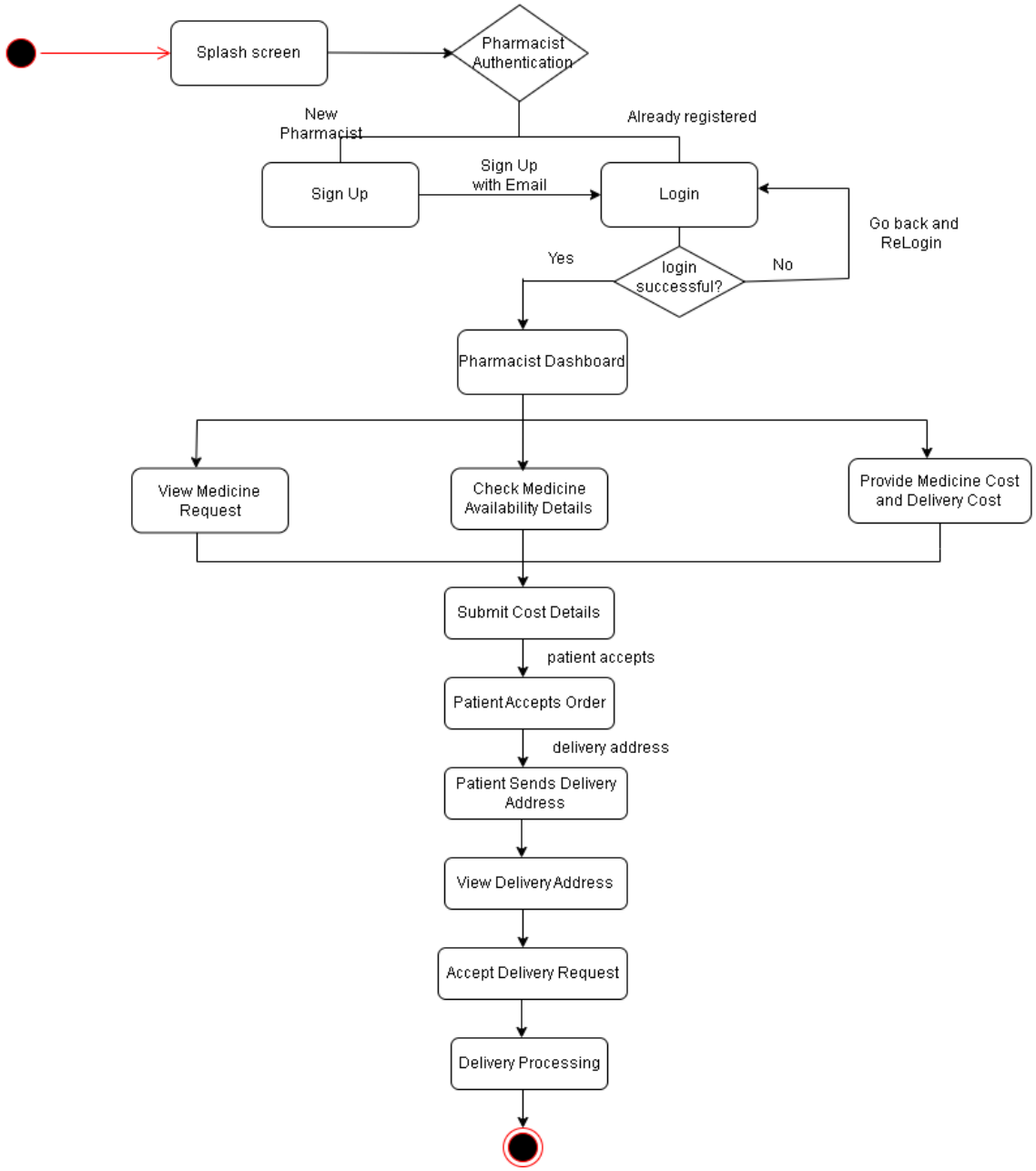
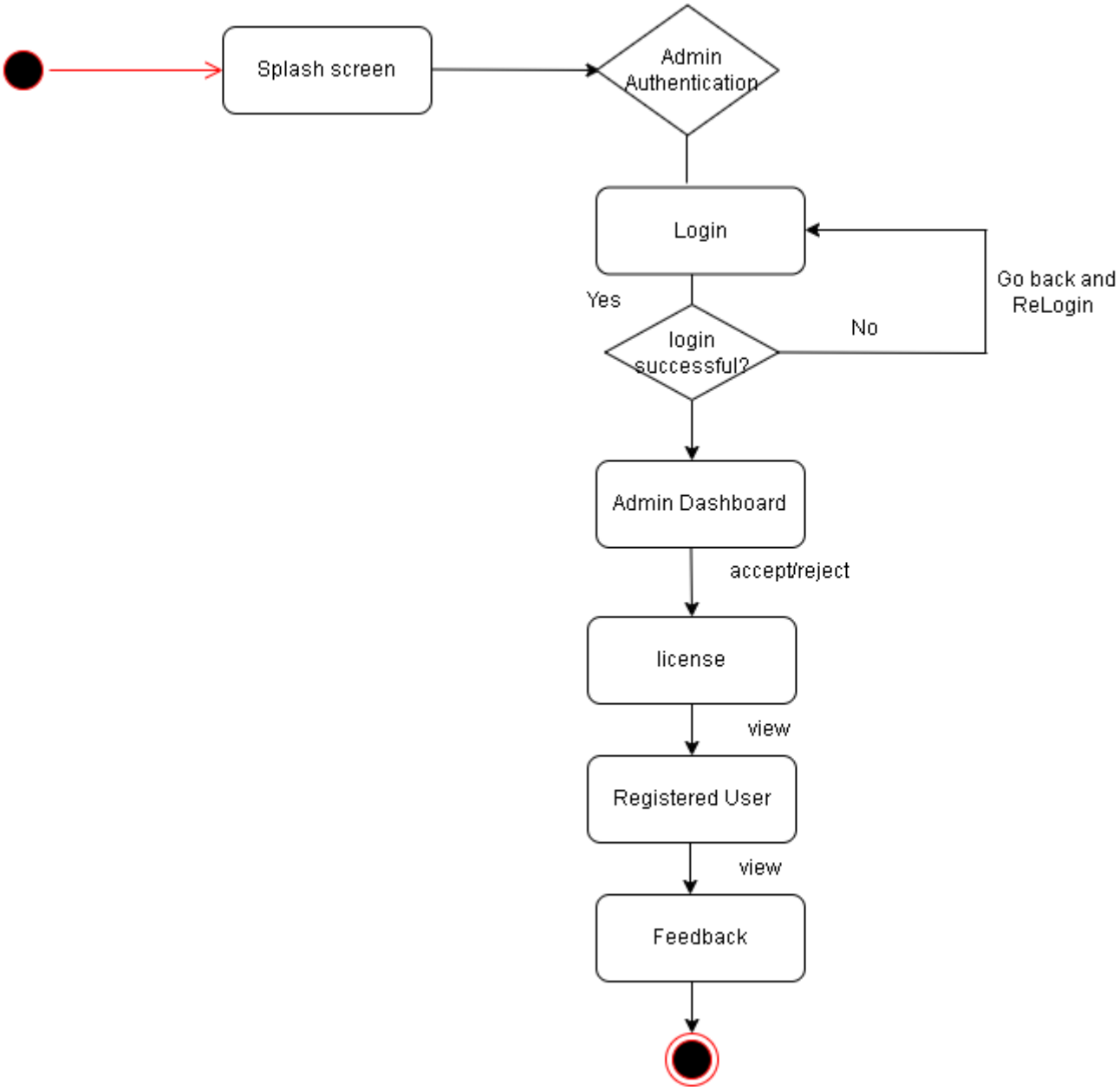

Splash screen
Pharmacist Authentication
New Pharmacist
Already registered
Sign Up
Sign Up with Email
Login
Go back and ReLogin
Yes
login successful?
No
Pharmacist Dashboard
View Medicine Request
Check Medicine Availability Details
Provide Medicine Cost and Delivery Cost
Submit Cost Details
patient accepts
Patient Accepts Order
delivery address
Patient Sends Delivery Address
View Delivery Address
Accept Delivery Request
Delivery Processing

Figure 3.4 State Diagram for pharmacist



3.3 Architectural Design

Architecture Design is concerned with understanding how a software system should be organized and designing the overall structure of that system . Architectural design is the first stage in the software design process . It is the critical link between design and requirements engineering ,as it identifies the main structural components in a system and the relationships between them. The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components .The smart Healthcare reminder application follows the Model-View-View Model(MVVM)architecture. This architecture divides the application into three main layers which help keep the code organized, makes the system easy to manage, and simplifies testing and maintenance.

1.Architecture Pattern: MVVM (Model-view-View Model)

Model: The model layer handles the data of the application. It stores, retrieves, and processes data from the database.

In this project, the model layer includes:

- User profile information
- Medication reminders
- Health measurement data(blood pressure, weight, blood sugar)
- Activity reminders(walking, water intake, eating meal)
- Appointment schedules
- Medicine Refill reminders
- Track intake records
- Chat details
- Payment details
- Feedback data
- Emergency contact details
- Chatbot query and response data
- Storing and retrieving data from Firebase Firestore

View: The view layer is the user interface of the application. It is the part that users interact with directly. It displays information and allows users to perform actions in the application.

The View layer of this project includes:

- Login and registration screens
- Home screen
- Doctor Dashboard
- Caregiver Dashboard
- Pharmacist Dashboard
- Medication reminder screen
- Health Measurement screen
- Activity reminder screen
- Appointment scheduling screen
- Pharmacy delivery screen
- Chatbot screen
- Chat Screen
- Payment Screen
- Track intake screen
- Feedback screen

- Emergency contact screen
- Profile section screen

View Model: The View Model act as a bridge between the Model and View layers. It manages the application logic and prepare data so it can be easily displayed in the user interface.

The functionality of this layer includes:

- Managing UI related data
- Fetching reminders and health data from Firebase
- Processing user actions such as adding or editing reminders
- Handling reminder scheduling and intake tracking
- Separates business logic from view

3.3.1 Architecture Pattern

Figure 5:Architecture Diagram

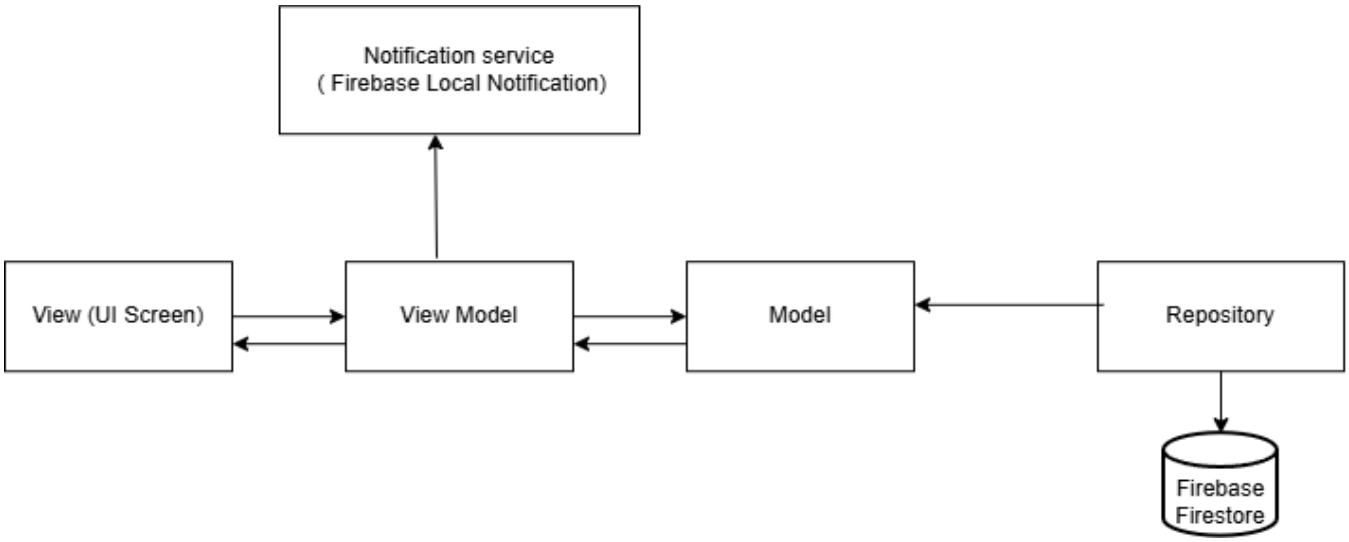


Figure 3.6 Architecture Diagram

3.3.2 Component structure

Component	Role
User Authentication module	Handles user registration, login and logout using Firebase authentication.
Medication Management module	Allows user to add, edit and manage medicine reminder and schedule.
Measurement module	Allows user to set reminder and record health measurement such as blood pressure, blood sugar, weight.

Activity Reminder module	Allows user to set reminder for daily physical activities like walking, drinking water, eating meal.
Appointment management module	Enables user to schedule and manage doctor appointment.
Chat Module	Allow the doctor and patient to communicate through a chat.The patient and caregiver can also communicate through chat.
Payment Module	Allow the doctor to send Easypaisa or JazzCash payment details to the relevant patient through chat after submitting a recommendation
Intake tracking module	Allows user to mark reminders as taken or missed and stores daily health records.
Profile Management module	Allows user to view and update their personal profile and health reminder history.
Pharmacy delivery module	Allow patients to send medicine requests to a pharmacist by entering the medicine name, quantity, pharmacy name, pharmacy address, and phone number. The pharmacist shall receive the request through the Pharmacy Dashboard.
Chatbot Assistance module	Helps users by answering basic health related questions, providing simple health tips and guiding them to manage reminders.
Feedback module	Allows user to submit feedback, complaints or queries about the system.
Emergency Contact module	Allows user to add, manage and quickly access Emergency contact such as doctor's PA name or phone number.
Admin management module	Allows admin to manage users and approve or reject license of user, view feedback and see system activities for control and maintenance.
Firebase Data module	Handles data storage and retrieval using Firebase Firestore and manages CRUD operation

View Model layer	Act as a bridge between the user interface and backend services

3.3.3 Component Diagram

Figure 6:Component Diagram

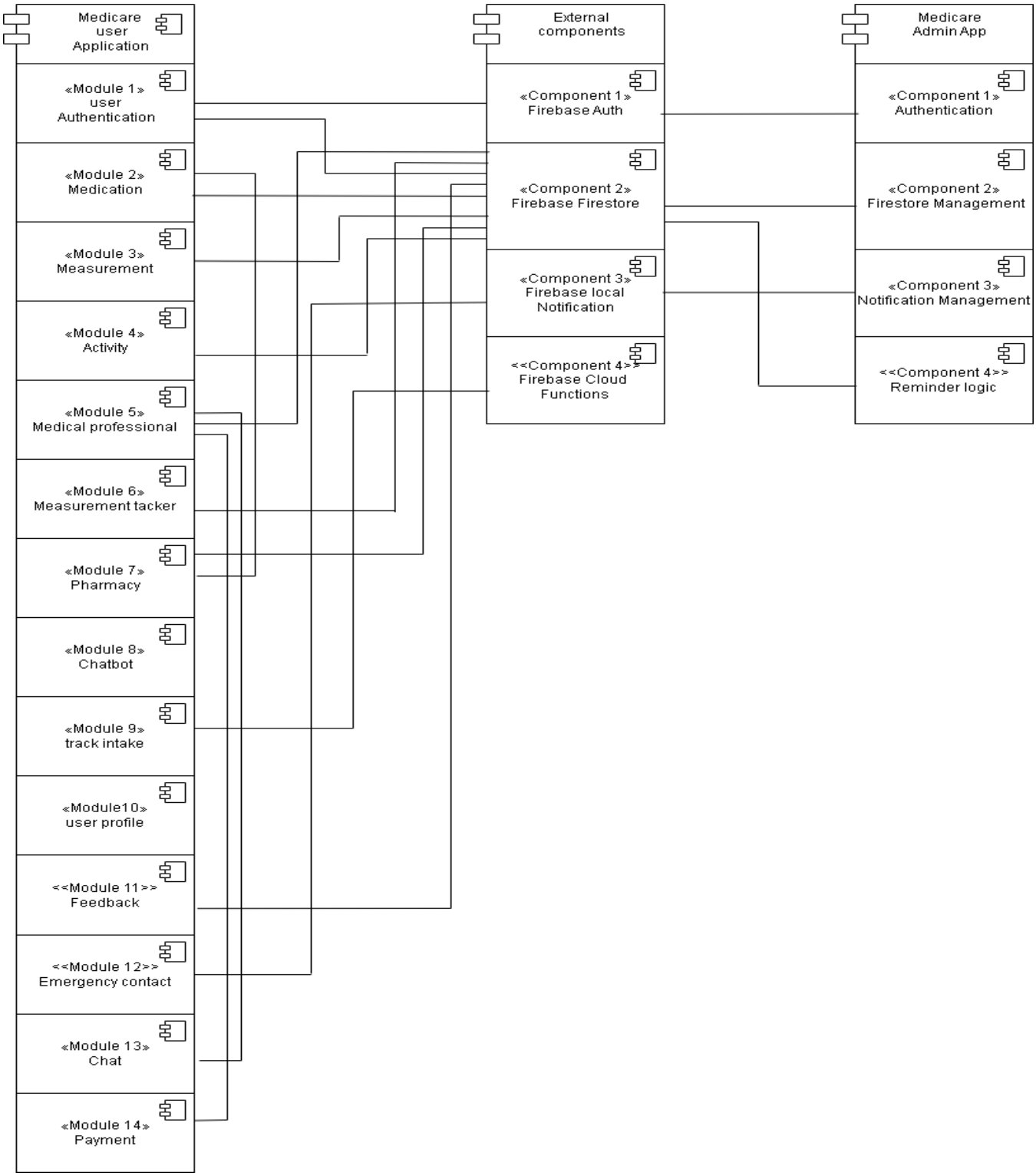


Figure 3.7 Component Diagram

3.4 Data Dictionary

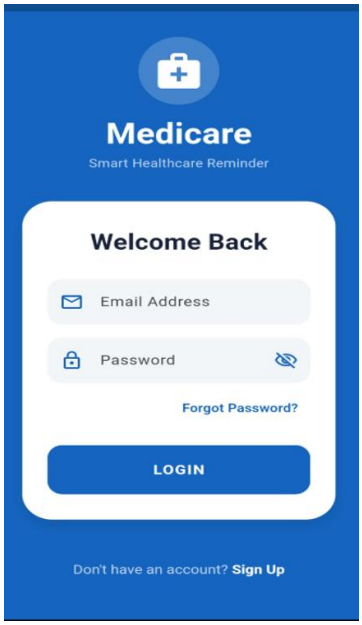
As we are using object-oriented approach so our data dictionary is comprised of classes, major classes are given below:

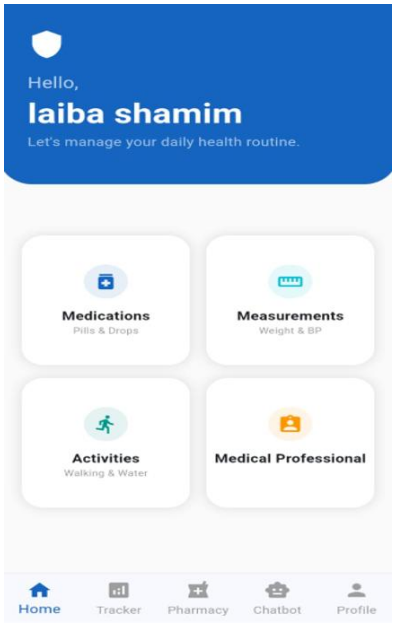
Terminology	Description
User	Class: Stores personal details such as name, age, date of birth, gender and manage medication,measurement,activities and appointments.
Medication	Class: Used to store medication details including medication name, dosage, frequency, start/end date, and reminder time.
Measurement	Class: Represents Reminders for health measurement such as blood pressure, weight or blood sugar. Stores Reminder type and scheduled time.
Activity	Class: Used to store activity reminders such as walking, drinking water, or meals. Store Reminder type and schedule time.
Appointment	Class: Stores doctor appointment details such as date, time, Doctor or clinic name.
Refill	Class: Represents Medicine Refill information such as medicine name and quantity.
pharmacy	Pharmacy Medicine Delivery allows patients to order required medicines, check availability and receive them at their specified delivery address.
Chatbot	Class: Acts as a virtual assistant that handles user queries related to health, reminders, and app usage by providing predefined responses and guidance and allow users to give feedback on the quality of its responses.
Notification	Class: Used to manage reminder notification for medication, activities, measurement, appointment and Refill
Feedback	Class: Used to store feedback such as complaint, suggestions or queries about the system.
Emergency contact	Class: Used to store emergency contact details such as name or phone numbe. It allows user to quickly access and contact important person.

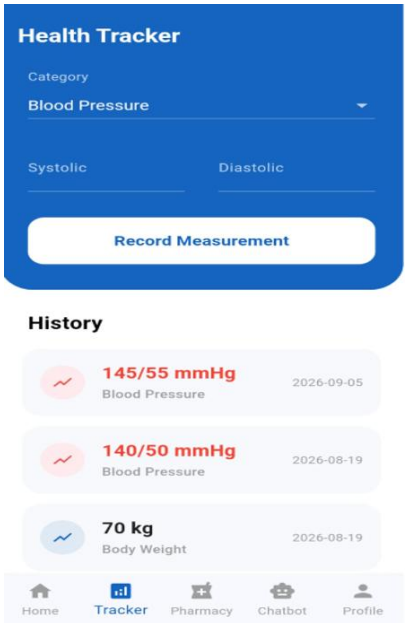
Firebase Authentication	Class: Used to interact with firebase Authentication for user login, signup.
Firebase Firestore	Class: Used to interact with firebase firestore to store and retrieve user data and reminders.
Document Snapshot	Class: Used to interact with documents retrieve from Firestore.

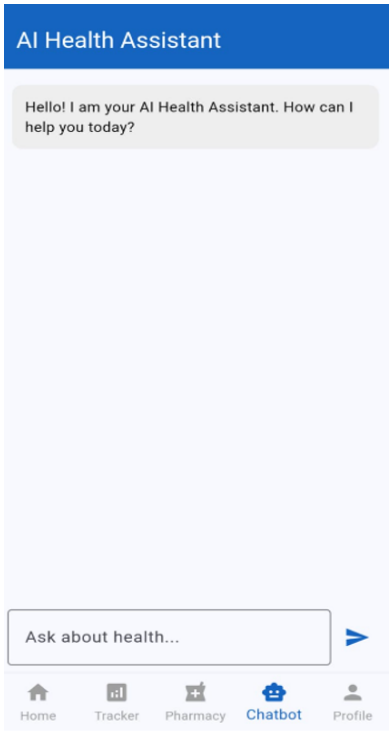
3.5 User Interface Design

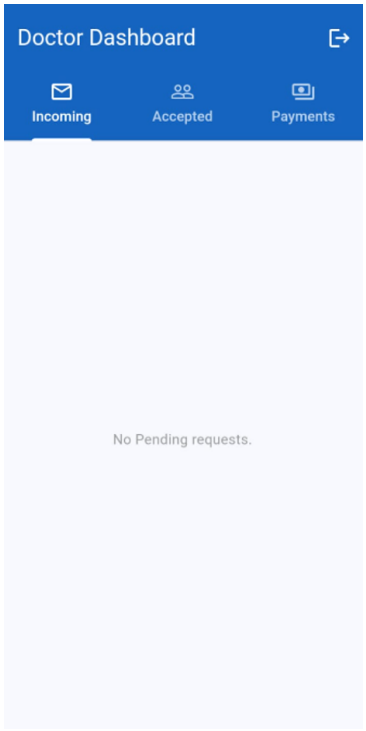
3.5.1 Screen Images

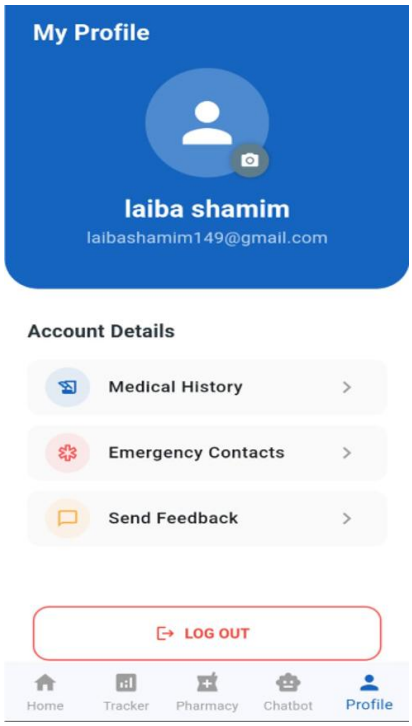












3.6 Design Decisions

In designing smart healthcare reminder mobile application, several key decision were made to ensure scalability, user-friendliness, maintainability and security while supporting healthcare-related reminders.

1.Design Methodology

Choice: Structured and modular approach

Reason: This approach divides the system into clear modules making the system easy to understand, maintain and extend in the future.

2.Architectural style

Choice: MVVM(Model, View, ViewModel)

Reason: This improves maintainability and scalability, allowing independent updates to the UI, logic, and back-end.

3.Database design

Choice: Firebase Firestore

Reason: Firestore supports hierarchical data storage which is suitable for managing dynamic data

4.Programming languages & Framework

Choice: Flutter for Android frontend and Firebase (Authentication, Firestore, Cloud Messaging and local notification) for backend services.

Frontend: Android(Flutter)

Backend: Firebase (Authentication, Firestore)

Reason: Flutter provides native android experience with strong performance, while Firebase offers reliable backend services.

5.Algorithmic Approach

Choice: CRUD operation with time- based scheduling logic.

Reason: CRUD operation are essential for creating, updating and deleting reminders and user data while time based logic ensures accurate scheduling and delivery of notification.

6.Security decisions

Choice: Firebase authentication with secure login options.

Reason: This ensures sensitive data is managed securely and reduces the risk of unauthorized access.

7.Design Tools

Choice: UML modelling tools such as draw.io.

Reason: These tools help visualize system structure through use case, class, sequence, state and improving system understanding.

3.7 :Summary

This chapter presents a detailed analysis of the system design , covering design considerations, architectural structure , and high level abstract models. It includes sequence, state , class, and component diagrams. The MVVM architecture pattern is used for its simplicity and ease of testing.

Chapter 4

Implementation

4. Implementation

This chapter describes the implementation part of the project before starting to create a project in Firebase make sure its package name matches the project package name created in android studio. This chapter will provide major logic algorithms to help build a complete app.

4.1 Algorithms

Name:	Algorithm 1 Splash Screen
Input:	App starts, user login status
Output:	User is sent to Home screen or Login screen
Steps:	1. App opens and splash screen is shown. 2. App checks if the user is already logged in. 3. If user is logged in, open Home screen. 4. If user is not logged in, open Login screen. 5. Splash screen closes

Name:	Algorithm 2 Sign Up
Input:	Name, email, date of birth, gender, password
Output:	New account is created, user goes to Login screen
Steps:	1. User opens Sign Up screen. 2. User enters name, email, date of birth, gender, and password. when user click Doctor, Caregiver, Pharmacist option it will enter license number or speciality and send request to admin for the approval and rejection. 3. App checks all fields are filled correctly. 4. If something is wrong, show error message. 5. If all correct, create the account. 6. Send a verification email to the user. 7. Take user to Login screen.

Name:	Algorithm 3 Login
Input:	Email and password

Output:	User is logged in and goes to Home screen
Steps:	1. User opens Login screen. 2. User enters email and password. 3. App checks that fields are not empty. 4. If empty, show error message. 5. App checks email and password are correct. 6. If email is not verified, ask user to verify first. 7. If correct, take user to Home screen. 8. If user forgot password, send a reset link to their email.

Name:	Algorithm 4 Patient Module
Input:	Patient login, health task details
Output:	Patient manages all health tasks in one place
Steps:	1. Patient logs in and opens Home screen. 2. Patient can add medication reminders. 3. Patient can set reminders for BP, sugar, and weight. 4. Patient can set reminders for daily activities. 5. Patient can book an appointment with a doctor. 6. Patient can view doctor recommendations. 7. Patient can mark tasks as taken or missed. 8. Patient can view health history in Profile.

Name:	Algorithm 5 Doctor Module
Input:	Doctor login, patient profile
Output:	Doctor views patient data and gives recommendations
Steps:	1. Doctor logs in with email and password. 2. Doctor opens the patient list. 3. Doctor selects a patient to view. 4. Doctor sees patient's medicines, measurements, and reminder history. 5. Doctor checks if patient is taking medicines on time.

	6. Doctor writes recommendations like medicine changes or checkup schedule. 7. Doctor decides if patient needs a caregiver or can manage alone. 8. Patient can see doctor's notes in their profile. 9. Doctor logs out when done.

Name:	Algorithm 6 In-App Chat
Input:	Approved request status, text message
Output:	Message exchange and payment detail sharing
Steps:	1. Doctor and caregiver approves patient request. 2.App updates status to Approved. 3. App enables Chat button for users. 4. User opens chat and types a message. 5. If any field is missing, display error message. 6. If valid, send and display message on screens.

Name:	Algorithm 7 Payment
Input:	Payment details (Doctor name, payment number, Transaction ID, receipt screenshot)
Output:	Payment marked as Approved or Rejected
Steps:	1. Doctor opens the Payments tab in the Doctor Portal. 2. App shows all pending patient payments. . 3. Doctor selects a payment to view the receipt screenshot and Transaction ID. 4.Doctor taps Approve or Reject. . 5. App updates the payment status in the database. 6. Patient sees the updated status on their screen.

Name:	Algorithm 8 Caregiver Module
Input:	Caregiver login, patient profile
Output:	Caregiver monitors and manages patient health tasks
Steps:	1. Caregiver logs in with email and password. 2. Caregiver opens assigned patient's profile. 3. Caregiver can see all active reminders of the patient. 4. Caregiver updates task status as taken or missed on behalf of patient. 5. Caregiver tracks patient's daily health progress. 6. Caregiver views patient's measurement records. 7. Caregiver makes sure patient does not miss any health task. 8. Caregiver logs out when done.

Name:	Algorithm 9 Medication Reminder
Input:	Medicine name, dosage, frequency, start date, end date, reminder time
Output:	Medication reminder is saved and notification is set
Steps:	1. User opens Medication screen. 2. User enters medicine name, dosage, frequency, and reminder time. 3. App checks all fields are filled. 4. If not filled, show error. 5. If filled, save the medication reminder. 6. Set a notification at the reminder time. 7. Show medicine in the reminder list. 8. User marks medicine as taken or missed when notification comes. 9. User can update or delete the reminder anytime.

Name:	Algorithm 10 Measurement Reminder
Input:	Measurement type (BP / Sugar / Weight), reminder time
Output:	Measurement reminder is saved and notification is set

Steps:	1. User opens Measurement screen. 2. User selects measurement type and sets reminder time. 3. App checks that type and time are selected. 4. If not selected, show error. 5. If selected, save the reminder. 6. Set a notification at the reminder time. 7. Show reminder in active list. 8. User can update or delete the reminder anytime.

Name:	Algorithm 11 Activity Reminder
Input:	Activity type (Water / Meal / Walk / Exercise), reminder time, frequency
Output:	Activity reminder is saved and notifications are set
Steps:	1. User opens Activity screen. 2. User selects activity type, reminder time, and frequency. 3. App checks all options are selected.

Name:	Algorithm 12 Medical Professional
Input:	(Patient, Doctor, Caregiver) connection request, Appointment Schedule
Output:	Patient connect with doctor or caregiver and appointments saved with reminder set
Steps:	1. User open the medical professional screen 2. User go to doctor tab enter doctor details, reasons/symptoms and send request 3. User go to caregiver tab enter the caregiver name and email and send request 4. App checks that all fields are filled 5. If any filed is empty shows error message 6. App sends the request and update the status to pending under requests and responses

	7. For Appointment, user select the appointment tab to view or manage schedule visits

Name:	Algorithm 13 Appointment Reminder
Input:	Appointment date, time, doctor or clinic name
Output:	Appointment is saved and reminder notification is set
Steps:	1. User opens Appointment screen. 2. User enters date, time, and doctor or clinic name. 3. App checks all fields are filled and date is in the future. 4. If not, show error. 5. If correct, save the appointment. 6. Set a reminder notification at the appointment time. 7. Show appointment in upcoming list. 8. User can update or delete the appointment anytime.

Name:	Algorithm 14 Refill Reminder
Input:	Medicine name, quantity
Output:	Refill reminder is saved, notification sent when medicine is low
Steps:	1. User opens medication screen. 2. User enters medicine name and current quantity. 3. App checks fields are filled. 4. If not, show error. 6. When quantity becomes low, send a notification to refill. 7. Show refill reminder in active list. 8. User can update or delete the refill reminder.

Name:	Algorithm 15 Measurement Tracker
Input:	Measurement type, values (BP readings / sugar level / weight)

Output:	Health value is saved and shown in history
Steps:	1. User opens Measurement Tracker screen. 2. User selects measurement type. 3. User enters the value (e.g. systolic and diastolic for BP). 4. App checks values are valid numbers. 5. If not valid, show error. 6. If valid, save the record with date and time. 7. Show record in health history list. 8. User can update or delete any record.

Name:	Algorithm 16 Pharmacy
Input:	Medicine name, Quantity, Delivery Address
Output:	Checked medicine stock, calculated the total price and placed the delivery order
Steps:	1. User opens Pharmacy screen. 2. User enters medicine name and required quantity 3. User taps check delivery availability. 4. App checks stock and delivery availability. 5. If out of stock app displays an error message. 6. If available, patient enters delivery address. 7. Pharmacist calculates medicine cost and delivery fee. 8.Patient confirms and places order using Cash on Delivery. 9.App saves order details and displays order confirmation to patient.

Name:	Algorithm 17 Chatbot
Input:	User's health question
Output:	AI gives a simple health answer or tip
Steps:	1. User opens Chatbot screen. 2. User types a health question. 3. Question is sent to the AI model.

	4. AI reads the question and finds the best answer. 5. If user shares a health value like high BP, AI gives a helpful tip. 6. AI reply is shown in the chat. 7. If AI does not understand, it shows a helpful default message. 8. Chat history stays on screen during the session.

Name:	Algorithm 18 Notification and Reminder
Input:	Reminder time set by user in any module
Output:	Notification is sent, status marked as Taken or Missed
Steps:	1. App checks all saved reminders. 2. At the set time, a notification is sent to the user. 3. Notification shows the task and reminder type. 4. If user marks it as Taken, status is saved as Taken. 5. If user ignores it, notification is repeated until user marked it. 6. Result is saved in Track Intake.

Name:	Algorithm 19 Track Intake
Input:	User response to reminder (Taken or Missed)
Output:	Daily health record is updated
Steps:	1. User opens Track Intake screen. 2. All today's reminders are shown. 3. User selects a reminder and marks it as Taken or Missed. 4. Status is saved with date and time. 5. User can see daily summary of all Taken and Missed tasks. 6. All records are saved and can be viewed in Profile History.

Name:	Algorithm 20 Profile Section
Input:	User profile info, feedback text, emergency contact details

Output:	Profile updated, feedback sent to admin, emergency contact saved
Steps:	1. User opens Profile screen. 2. User can see their personal details like name and email. 3. User goes to History to see all Taken and Missed records. 4. User goes to Feedback and writes a suggestion or complaint. 5. App checks feedback is not empty. 6. If not empty, feedback is saved and sent to admin. 7. User adds emergency contact. 8. App checks name and number are valid. 9. Emergency contact is saved for quick access in emergencies.

Name:	Algorithm 21 Admin Module
Input:	Admin login credentials
Output:	The admin can view registered users. User can submit feedback which can be viewed by the admin.
Steps:	1. Admin enters email and password to login. 2. Admin opens Admin Dashboard. 3. Admin can see list of all registered users. 4.Admin can approve and reject license of doctor. 5. Admin can view all feedback sent by users. 6. Admin can view emergency contact details saved by users. 7. Admin logs out when done.

4.2 External APIs/SDKs

Medicare uses firebase services for its main external services. Firebase authentication is used for user registration while cloud Firestore is used for data management and Firebase Cloud messaging for push notification. Flutter Local Notification is used to display scheduled or instant notifications on a user's device All these services are included via Gradle dependencies and are used as built-in-component within the android project.

4.3 Code Repository

Git repository Link: https://github.com/laibashamim189-dotcom/medicaree. The source code of the project is stored in a git repository ,which contains all the modules, features of the Medicare. It is a public repository; entire code is there and will be updated on a daily basis as we test our app and improve it.

4.4 Summary

This chapter outlined almost all major components algorithms although not all are included here in this chapter, as there's a lot of code. But one can get the understanding and working and flow of the things and understand the things from the repository.

Chapter 5

Testing and evaluation

5: Introduction

This chapter focuses on testing and evaluation of the project. The validation and verification of different components will be considered. The project is still under development, mostly testing is done manually, no automated scripts have been followed yet.

5.1 Unit Testing (UT)

Table 51:Unit Test Case 1

5.1.1 Test Case 1

Field	Details
Testcase ID	UT1
Title	User Authentication (Login & Registration)
Description	This test verifies that a patient, caregiver, doctor and pharmacist can register a new account and successfully log into the Smart Healthcare Reminder system.
objective	To confirm correct authentication ,user creation and secure access to the system.
Driver/precondition	Firebase Authentication initialized
Test steps	1. Register new user with email/password 2. Store user in Firestore 3. Login using credentials 4. Verify access to dashboard
Input	Email, password, personal details
Expected Results	User should be registered and successfully logged in
Actual Results	User created and login successful
Remarks	Pass

Table 52:Unit Test Case 2

5.1.2 Test Case 2

Field	Details
Testcase ID	UT2
Title	Medication Reminder Creation
Description	The patient adds medication and system schedules reminders.
objective	Verify that medication data is stored and reminder is generated.
Driver/precondition	Patient is logged in
Test steps	1. Open Medication module 2. Add medicine details 3. Save medication 4. Check reminder notification
Input	Medicine name, dosage, time
Expected Results	Medication saved and reminder notification scheduled
Actual Results	Medication stored and notification triggered
Remarks	Pass

Table 53:Unit Test Case 3

5.1.3 Test Case 3

Field	Details
Testcase ID	UT3
Title	Health Measurement Recording (BP, Sugar, Weight)
Description	Patient records health measurement and the system stores them in history.
objective	Ensure measurement tracking works correctly.
Driver/precondition	Patient logged in
Test steps	1. Open Measurement module 2. Enter BP/sugar/weight 3. Save record 4. View history
Input	BP, sugar level, weight values
Expected Results	Measurement stored and displayed in history
Actual Results	Data saved successfully
Remarks	Pass

Table 54:Unit Test Case 4

5.1.4 Test Case 4

Field	Details
Testcase ID	UT4
Title	Appointment Reminder

Description	Patient will reminder an appointment with a doctor.
objective	Ensure the appointment module works correctly.
Driver/precondition	Doctor list available patient logged in
Test steps	1. Open Appointment module 2. Select doctor 3. Choose date/time 4. Notification will remind
Input	Doctor name, speciality, date, time
Expected Results	Appointment should be saved.
Actual Results	Appointment saved successfully
Remarks	Pass

Table 55:Unit Test Case 5

5.1.5 Test Case 5

Field	Details
Testcase ID	UT5
Title	Doctor Recommendation Submission
Description	Doctor can enter and save a recommendation for the selected patient.
Objective	To ensure that doctor recommendation is correctly validated and stored in a system.
Driver/precondition	Doctor is logged in and has access to an approved patient profile.

Test steps	1.Open patient request, select accept and review it. 2.Open Recommendation section 3.Enter recommendation 4.Submit recommendation 5. Validate input 6. Save recommendation 7.Verify successful saving
Input	Doctor recommendation details
Expected Results	The recommendation is successfully validated and stored for the selected patient.
Actual Results	Doctor recommendation was successfully submitted and stored.
Remarks	Pass

Table 56:Unit Test Case 6

5.1.6 Test Case 6

Field	Details
Testcase ID	UT6
Title	Pharmacy Medicine Request
Description	This test verifies that patient can create and submit medicine request to the patient.
objective	To ensure that medicine request details are correctly validated and stored in the system.
Driver/precondition	Patient is logged in and the Pharmacy module is available.
Test steps	1.Open Pharmacy section 2. Enter medicine name and quantity 3. Enter pharmacy details 4.Submit medicine request 5.Validate input 6.Save request 7.Verify successful submission.

Input	Medicine name, quantity, Pharmacy name, Pharmacy Address and phone number.
Expected Results	The medicine request is successfully validated and stored in the system for the pharmacist.
Actual Results	Medicine request was successfully submitted and stored.
Remarks	Pass

Table 57:Unit Test Case 7

5.1.7 Test Case 7

Field	Details
Testcase ID	UT7
Title	Doctor-Patient Chat
Description	This test verifies that an approved patient and doctor can send and store messages.
objective	To ensure that chat messages are correctly sent and stored in the system.
Driver/precondition	Patient and doctor are logged in and the doctor has approved the patient request.
Test steps	1.Open Chat section 2. Enter message 3.Send message 4.Validate message 5.Store message in Firestore 6.Verify successful sending and storage.
Input	Chat message
Expected Results	The message is successfully sent and stored in the system for selected doctor and patient.
Actual Results	Message was successfully sent and stored.

Remarks	Pass

Table 58:Unit Test Case 8

5.1.8 Test Case 8

Field	Details
Testcase ID	UT8
Title	AI Chatbot Health Guidance
Description	Patient uses chatbot for basic health advice.
objective	Ensure chatbot provides correct responses
Driver/precondition	Internet connection active
Test steps	1. Open chatbot 2. Ask health question 3. Receive response
Input	Health query
Expected Results	Relevant health guidance response
Actual Results	Response generated successfully
Remarks	Pass

Table 59:Unit Test Case 9

5.1.9 Test Case 9

Field	Details

Testcase ID	UT9
Title	Feedback Submission
Description	This test verifies that user can submit feedback and feedback is successfully stored in system.
objective	To verify that the feedback submission function correctly validates and saves user feedback.
Driver/precondition	User is logged in and the feedback module is available.
Test steps	1. User opens feedback section 2. Enters feedback 3. Submits feedback 4.Validate input save feedback to firestore and verify successful submission.
Input	User feedback text
Expected Results	Feedback is validated and successfully stored in Firestore, and confirmation message is displayed.
Actual Results	Feedback was successfully submitted and stored in Firestore.
Remarks	Pass

Table 60:Unit Test Case 10

5.1.8 Test Case 10

Field	Details
Testcase ID	UT10
Title	Emergency Contact Save and Call
Description	This test verifies that user can save an emergency contact details and can initiate a call using the saved phone number.

objective	To verify that emergency contact provide quick access to the saved phone number.
Driver/precondition	User is logged in and phone permission is available.
Test steps	1. Open emergency contact section 2. Enter contact name and phone number 3. Save contact 4. Verify saved contact 5. Select contact 6. Tap call option 7. Verify phone call screen opens
Input	Emergency contact name and phone number
Expected Results	Contact should be saved successfully and the call option should open the device's calling interface with the saved number.
Actual Results	Contact was saved successfully and the calling interface opened with the saved number.
Remarks	Pass

5.2 Functional Testing (FT)

Table 61:Functional Test Case 1

5.2.1 Test Case 1

Field	Detail
Testcase ID	FT1
Title	Health Modules(Medication,Measurement, activities, appointment and refill)
Description	This test verifies that the patient can successfully create and manage health-related modules including medication reminders, health measurements, appointments, refill reminders, and daily activities in the system.
Objective	To ensure that all health modules correctly store data in Firestore and trigger scheduled reminders.

Driver/Precondition	Patient is authenticated, Firestore initialized,all modules accessible
Test steps	1. Add medication reminder 2. Add measurement record 3. Book appointment 4. Set refill reminder 5. Add activity reminder 6. Verify data stored in Firestore
Input	Medicine details, BP/sugar/weight values, appointment info, refill data, activity type
Expected Results	All records should be stored successfully and reminders scheduled properly
Actual Result	All modules data saved successfully in Firestore
Remark	Pass

Table 62:Functional Test Case 2

5.2.2 Test Case 2

Field	Detail
Testcase ID	FT2
Title	User Authentication (Login & Registration)
Description	This test verifies that a patient can register a new account and successfully log into the Smart Healthcare Reminder system.
Objective	To confirm correct authentication, user creation, and secure access to the system.
Driver/Precondition	Firebase Authentication initialized
Test steps	1. Register new user with email/password 2. Store user in Firestore 3. Login using credentials 4. Verify access to dashboard

Input	Email, password, personal details
Expected Results	User should be registered and successfully logged in
Actual Result	User created and login successful
Remark	Pass

Table 63:Functional Test Case 3

5.2.3 Test Case 3

Field	Detail
Testcase ID	FT3
Title	Sending Request to doctor and Patient Profile Review
Description	This test verifies the complete appointment process in which a patient books an appointment and the doctor views the patient's profile after the appointment acceptance.
Objective	To verify that after accepting the request provides authorized access to the patient's health profile for the doctor.
Driver/Precondition	Patient and doctor accounts are registered and authenticated.
Test steps	1. Patient opens Doctor in medical professional section. 2. Enter Doctor name, Email, speciality and Reason/symptoms to the doctor. 3. Doctor accept and review request of incoming patient. 5. Doctor views profile and health history of accepted patients.
Input	Doctor Name, Email, speciality, Reason/symptoms
Expected Results	Appointment should be booked successfully and the doctor should be able to view the authorized patient's profile.
Actual Result	Appointment was booked and doctor successfully viewed the patient's profile.
Remark	Pass

Table 64:Functional Test Case 4

5.2.4 Test Case 4

Field	Detail
Testcase ID	FT4
Title	Doctor Recommendation
Description	This test verifies that doctor can analyze patient health information And provide recommendation such as medication,measurement frequency,activity and caregiver support.
Objective	To verify that doctor recommendations are successfully provided and made available to the patient.
Driver/Precondition	Doctor has authorized access to the patient's profile.
Test steps	1. Doctor accept and review patient request. 2. Recommendation screen open and doctor provide recommendation for medication,measurement frequency,activities and caregiver support 4. Selects caregiver required or not required 5. Submits recommendation 7. Patient views recommendation
Input	Patient health data and doctor recommendations
Expected Results	Doctor should be able to save recommendations and the patient should be able to view them.
Actual Result	Recommendations were successfully saved and displayed to the patient.
Remark	Pass

Table 65:Functional Test Case 5

5.2.5 Test Case 5

Field	Details
Testcase ID	FT5
Title	Pharmacy Medicine Stock and Order
Description	This test verifies that the pharmacy module correctly checks medicine stock, calculates the total price andd processes the medicine order.
Objective	To confirm that the individual pharmacy functions work correctly for checking stock, calculating cost, and placing an order.
Driver/Precondition	Pharmacy module is available and medicine stock information is stored in the system.
Test steps	1. User opens pharmacy screen 2. User enters medicine name and required quantity 3. System checks medicine stock 4. System checks delivery availability 5. If medicine is out of stock, system displays an error message 6. If medicine is available, system calculates medicine cost and delivery fee 7. User places the order 8. System saves order details and displays confirmation
Input	Medicine name, quantity, and delivery address
Expected Results	System should correctly check medicine availability, calculate the total cost, and successfully save the order details.

Actual Result	Medicine stock was checked, total cost was calculated, and order details were successfully saved.
Remark	Pass

Table 66:Functional Test Case 6

5.2.6 Test Case 6

Field	Detail
Testcase ID	FT6
Title	Profile & Health History Management
Description	This test verifies that the patient can view and manage their profile, health history and feedback section.
Objective	To confirm that all user history (medication, measurements, activities, appointments) is correctly displayed and stored.
Driver/Precondition	Patient logged in, data available in Firestore
Test steps	1. Open profile section 2. View health history 3. Submit feedback 4. Verify data retrieval from Firestore
Input	User profile data and feedback text
Expected Results	Profile and history should be displayed correctly
Actual Result	Profile data and history shown successfully
Remark	Pass

Table 67:Functional Test Case 7

5.2.7 Test Case 7

Field	Details
Testcase ID	FT7
Title	Doctor, Pharmacist, Nurse Medical License Verification
Description	This test verifies that a user can enter their medical license number and specialty in the sign-up screen and submit the verification request to Firestore.
objective	To confirm that the license information is validated and the user verification request is successfully stored with Pending status.
Driver/precondition	Firebase Authentication and Firestore are configured, and the user entered the valid information.
Test steps	1. Open License Verification screen 2. Enter medical license, Drug license number, pharmacist license number and nurse license number. 3. Enter speciality 4. System validates the license number field 5. System saves information in the corresponding Firestore collection 6. System sets verification status as Pending.
Input	Medical license number, Drug license number, pharmacist license number, nursing license number, speciality
Expected Results	User information should be successfully saved in Firestore with the verification status Pending and a message should be displayed.

Actual Results	User information was successfully saved with Pending status
Remarks	Pass

Table 68:Functional Test Case 8

5.2.8 Test Case 8

Field	Details
Testcase ID	FT8
Title	Verify Doctor, pharmacist and nurse Login Based on License Approval Status
Description	This test verifies that the doctor is redirected to the relevant Dashboard when the admin approves the license while a rejected is shown the rejection status.
objective	To ensure that the system correctly controls user access based on the license verification status.
Driver/precondition	User license verification request is submitted. Admin has approved or rejected the license.
Test steps	1. Admin reviews the license request. 2. Admin approves the license. 3. User log in to the application. 4. System checks the doctor's verification status. 5. If license is accepted doctor is redirected to the Dashboard.. 6. Repeat the test with admin rejecting the license. 7. User logs in again and system checks the status. 8. If status is Rejected the rejection message/status is displayed to the doctor.
Input	User email/password and license verification status

Expected Results	If admin approves the license, the users status becomes active and then redirected to the Dashboard after login. If admin rejects the license, the user is not given access to the Dashboard and the rejected status/message is shown.
Actual Results	Approved user was redirected to the relevant Dashboard while rejected one was shown the rejection status.
Remarks	Pass

Table 69:Functional Test Case 9

5.2.9 Test Case 9

Field	Detail
Testcase ID	FT9
Title	Patient–Doctor Chat Functionality
Description	This test verifies the complete chat functionality between an approved patient and doctor.
Objective	To ensure that the patient and doctor can successfully communicate through their respective dashboards.
Driver/Precondition	Patient and doctor are registered and logged in and the doctor has approved the patient's request.
Test steps	1. Patient opens the doctor section. 2. Patient selects the approved doctor. 3. Patient opens the Chat option and sends a message. 5. Doctor opens the patient chat from the Doctor Dashboard. 6. Doctor views the patient's message. 7. Doctor sends a reply. 8. Patient views the doctor's reply.
Input	Patient and doctor messages

Expected Results	Chat option should be available for the approved patient-doctor relationship and both users should be able to send and receive messages successfully.
Actual Result	Patient and doctor successfully exchanged messages through the chat feature.
Remark	Pass

Table 70:Functional Test Case 10

5.2.10 Test Case 10

Field	Detail
Testcase ID	FT10
Title	Patient–Caregiver Chat Functionality
Description	This test verifies the complete chat functionality between approved patient and caregiver.
Objective	To ensure that patient and caregiver can successfully communicate through their respective dashboard.
Driver/Precondition	Patient and Caregiver are registered and logged in and caregiver has approved the patient's request.
Test steps	1. Patient opens the caregiver section. 2. Patient selects the approved caregiver. 3. Patient opens the Chat option and sends a message. 5. Caregiver opens the patient chat from the Caregiver Dashboard. 6. Caregiver views the patient's message. 7. Caregiver sends a reply. 8. Patient views the Caregiver's reply.
Input	Patient and Caregiver messages

Expected Results	Chat option should be available for the approved patient-caregiver relationship and both users should be able to send and receive messages successfully.
Actual Result	Patient and doctor successfully exchanged messages through the chat feature.
Remark	Pass

Table 71:Functional Test Case 11

5.2.11 Test Case 11

Field	Detail
Testcase ID	FT11
Title	Doctor Payment Functionality
Description	This test verifies the complete payment process and the approval or rejection of payment.
Objective	To ensure that the patient submit payment details and the doctor can verify the payment.
Driver/Precondition	Doctor has submitted a recommendation and provide Easypaisa/ Jazzcash number to the patient through the chat.
Test steps	1. Patient opens the Pay Doctor section. 2. Patient enters doctor name and payment number. Patient enters transaction ID.Patient uploads payment receipt screenshot. 3. Patient submits payment. 4. System displays PENDING status. 5. Doctor opens the Payment section. 6. Doctor reviews the transaction details and receipt. 7. Doctor selects APPROVE or REJECT. 8. Patient checks the updated payment status.
Input	Doctor name, payment number, transaction ID, receipt screenshot, and approval/rejection

Expected Results	Payment should be submitted successfully with Pending status. After doctor verification, the patient should see Approved or Rejected status.
Actual Result	Payment should be submitted successfully with Pending status. After doctor verification, the patient should see Approved or Rejected status.
Remark	Pass

Table 72:Functional Test Case 11

5.2.12 Test Case 12

Field	Detail
Testcase ID	FT12
Title	Pharmacist Medicine and Delivery Management Functionality
Description	This test verifies the complete medicine request and delivery process between the patient and pharmacist.
Objective	To ensure that the patient can request medicine and the pharmacist can manage the request and delivery.
Driver/Precondition	Patient and pharmacist are registered and logged in.
Test steps	1. Patient opens the Pharmacy section. 2. Patient enters medicine name and quantity. 3. Patient enters pharmacy name, address, and phone number and sends the request. 4. Pharmacist views the request and enters medicine and delivery cost. 6. Patient accepts the cost and submits delivery address. 7. Pharmacist views the delivery address. 8. Pharmacist accepts the delivery request.
Input	Medicine name, quantity, pharmacy details, medicine cost, delivery cost, and delivery address

Expected Results	The complete medicine request, cost confirmation, delivery address submission, and delivery acceptance process should work successfully.
Actual Result	Medicine request, cost confirmation, delivery address, and delivery acceptance were successfully processed.
Remark	Pass

5.3 Integration Testing (IT)

Table 73:Integration Test Case 1

5.3.1 Test Case 1

Field	Details
Testcase ID	IT1
Title	Authentication and Role-Based Access Integration
Description	This test verifies integration between firebase Authentication and role-bases access for patient, caregiver, doctor and pharmacist.
Objective	To confirm that each authenticated user is redirected to the correct dashboard according to their assigned role.
Driver/Precondition	Firebase Authentication and user role information are configured.
Test steps	1. User enters email and password 2. Firebase validates credentials 3. System retrieves user role 4. System redirects user to appropriate dashboard 5. Verify dashboard access
Input	Email, password and user role

Expected Results	Patient should access Patient Dashboard, caregiver should access Caregiver Dashboard and doctor should access Doctor Dashboard and Pharmacist should access Pharmacist Dashboard
Actual Result	Each user successfully redirected to the correct dashboard according to their role.
Remark	Pass

Table 74:Integration Test Case 2

5.3.2 Test Case 2

Field	Details
Testcase ID	IT2
Title	Reminder, Intake Status and History Integration
Description	This test verifies the integration between reminders, intake status and patient history.
Objective	To confirm that when a patient marks a reminder as taken or missed, the status is stored and reflected in the history.
Driver/Precondition	Patient is logged in and an active reminder exists.
Test steps	1. Patient receives reminder notification 2. Opens reminder 3. Marks task as Done or Dismiss. 4.Mark as patient or caregiver 5. System stores the status 6. Patient opens History 7. Verifies intake record

Input	Reminder and intake status
Expected Results	The selected intake status should be stored correctly and displayed in the patient's history.
Actual Result	Intake status was successfully stored and displayed in history.
Remark	Pass

Table 75:Integration Test Case 3

5.3.3 Test Case 3

Field	Details
Testcase ID	IT3
Title	Doctor Recommendation and Caregiver Integration
Description	This test verifies the integration between the doctor's caregiver recommendation, caregiver-patient management.
Objective	To confirm that when the doctor recommends caregiver support the patient can involve a caregiver to manage healthcare tasks.
Driver/Precondition	Doctor has reviewed the patient's profile and caregiver support is required.
Test steps	1. Doctor reviews patient condition 2. Doctor selects caregiver required 3. Saves recommendation 4. Patient views recommendation 5. Patient links/adds caregiver 6. Caregiver logs in 7. Caregiver accesses patient tasks

Input	Doctor recommendation and caregiver information
Expected Results	Caregiver support status should be saved and the authorized caregiver should be able to access and manage patient tasks.
Actual Result	Doctor recommendation was saved and the linked caregiver successfully accessed patient tasks.
Remark	Pass

Table 76:Integration Test Case 4

5.3.4 Test Case 4

Field	Details
Testcase ID	IT4
Title	Caregiver and Patient Reminder Integration
Description	This test verifies that an authorized caregiver can manage the patient's healthcare reminders on the patient's behalf.
Objective	To confirm that caregiver updates permitted patient reminders are reflected in the patient's account.
Driver/Precondition	Caregivers are linked with the patient and has permission to manage healthcare tasks.
Test steps	1. Caregiver logs in 2. Opens linked patient profile 3. Adds or updates a reminder 4. Saves changes 5. Patient opens reminder section 6. Verifies updated reminder

Input	Patient reminder details
Expected Results	Changes made by the caregiver should be stored and reflected in the patient's reminder section
Actual Result	Reminder changes were successfully reflected in the patient's account.
Remark	Pass

Table 77:Integration Test Case 5

5.3.5 Test Case 5

Field	Details
Testcase ID	IT5
Title	AI Chatbot Integration with System
Description	Tests integration of chatbot with user input and response system.
Objective	Ensure Chatbot process queries and returns responses correctly.
Driver/Precondition	Internet connection active
Test steps	1. Open chatbot 2. Enter query 3. Receive response
Input	Health-related query

Expected Results	Relevant response displayed
Actual Result	Response generated correctly
Remark	Pass

Table 78:Integration Test Case 6

5.3.6 Test Case 6

Field	Details
Testcase ID	IT6
Title	Doctor License Submission and Admin Approval Integration
Description	This test verifies the integration between the Doctor License Verification module and the Admin License Approval module.
Objective	To verify that the license submitted by the doctor is received by the admin and can be approved.
Driver/Precondition	Doctor will registered Firebase Authentication and Firestore are configured, and admin has access to the verification requests.
Test steps	1. Doctor opens signup screen. 2. Doctor enters license number and speciality. 3. System saves the request in Firestore with PENDING status. 4. Admin opens License Approval screen. 5. Admin views the submitted doctor request. 6. Admin reviews the license details and clicks Approve. 7. System updates the doctor's status to active in Firestore. 8. Doctor logs in and system checks the updated status.

	9. Doctor is redirected to the Doctor Portal.
Input	Doctor license information and admin approve action
Expected Results	Doctor's verification request should successfully transferred from the Doctor module to the Admin module through Firestore. After admin approval the doctor should be redirected to the Doctor Dashboard after login
Actual Result	License request was successfully submitted, reviewed by admin and the doctor was redirected to the Doctor Dashboard.
Remark	Pass

Table 79:Integration Test Case 7

5.3.7 Test Case 7

Field	Details
Testcase ID	IT7
Title	Admin License Rejection and Doctor Login Integration
Description	This test verifies that the rejected license status from the Admin module is correctly reflected when the doctor logs into the application.
Objective	To verify that a doctor with a Reject license status cannot access the Doctor Portal and is shown the rejection status.
Driver/Precondition	Doctor has submitted a license verification request, the request has Pending status and admin is logged in.
Test steps	1. Doctor submits license information. 2. System saves the request with Pending status.

	3. Admin opens the License Approval screen. 4. Admin reviews the doctor's license details. 5. Admin clicks Reject. 6. System updates the doctor status to Rejected in Firestore. 7. Doctor logs into the application. 8. System checks the doctor's verification status. 9. System displays the Rejected status/message to the doctor.
Input	Doctor license information and admin Reject action
Expected Results	The doctor's status should be changed from Pending to Rejected in Firestore. When the doctor logs in, the system should display the Rejected status and should not allow access to the Doctor Dashboard.
Actual Result	Doctor status was successfully changed to Rejected and the rejection status was displayed during login.
Remark	Pass

Table 80:Integration Test Case 8

5.3.8 Test Case 8

Field	Detail
Testcase ID	IT8
Title	Patient–Doctor Chat Integration
Description	This test verifies the integration between the Patient Dashboard, Doctor Dashboard, and Firebase/Firestore chat system.
Objective	To verify that messages sent by the patient are received by the doctor and doctor replies are received by the patient.
Driver/Precondition	Patient and doctor are logged in, doctor has approved the patient's request, and Firebase/Firestore is connected.

Test steps	1. Doctor approves the patient request. 2. Chat option becomes available on both dashboards. 3. Patient sends a message. 4. Message is stored in Firebase/Firestore. 5. Doctor receives the message and sends a reply. 7. Reply is stored in Firebase/Firestore. 8. Patient receives the reply.
Input	Patient and doctor messages
Expected Results	Patient and doctor messages should be correctly transferred and synchronized between both dashboards through Firebase/Firestore.
Actual Result	Messages were successfully stored and exchanged between the patient and doctor.
Remark	Pass

Table 81:Integration Test Case 9

5.3.9 Test Case 9

Field	Detail
Testcase ID	IT9
Title	Doctor Payment and Verification Integration
Description	This test verifies the integration between Doctor Recommendation, Patient Payment, Doctor Payment Verification, and Firebase/Firestore.
Objective	To verify that payment information submitted by the patient reaches the doctor and the doctor's decision is reflected on the patient's dashboard.
Driver/Precondition	Doctor has submitted a recommendation and provided payment details. Patient and doctor are logged in, and Firebase/Firestore is connected.

Test steps	1. Doctor submits recommendation. 2. Doctor sends Easypaisa/JazzCash number to patient through chat. 3. Patient submits transaction ID and receipt screenshot. 4. System saves payment with Pending status. 5. Doctor opens Payment section and reviews payment details and approves or rejects the payment. 7. System updates payment status in Firebase/Firestore. 8. Patient views the updated payment status.
Input	Recommendation, payment number, transaction ID, receipt screenshot, and doctor approval/rejection
Expected Results	Payment should move from Pending to Approved or Rejected and the updated status should be correctly displayed to the patient.
Actual Result	Payment details were successfully transferred, verified by the doctor, and the updated status was displayed to the patient.
Remark	Pass

5.4 Performance Testing (PT)

Performance testing is conducted to evaluate the system's response time, scalability and stability under load. The Smart Healthcare Reminder system must handle multiple users performing activities like adding reminders, coordination with doctor, caregiver and pharmacist and accessing data without delay.

Table 82:Performance Test Case 1

5.4.1 Test Case 1

Testcase ID	PT1
Requirement ID	R1 (User Authentication)
Title	User Login Response Time

Description	Tests how quickly the system logs in a user under normal and high load conditions
Objective	Ensure fast and efficient login performance.
Driver/precondition	Internet active, Firebase initialized
Test steps	1. Open login screen 2. Enter credentials 3. Tap login 4. Measure response time
Input	Valid email and password
Expected Results	Login should complete within 2–3 seconds
Actual Result	Login completed in 2 seconds
Remarks	Pass

Table 83:Performance Test Case 2

5.4.2 Test Case 2

Testcase ID	PT2
Title	Medication,Measurement,Appointment and Activity Data Storage Performance
Description	Measures time taken to store and schedule reminder.
Objective	Ensure quick database write and scheduling process.
Driver/precondition	Patient logged in
Test steps	1. Add details 2. Save data 3. Measure response
Input	Medicine name, Measurement ,Activity type, dosage, time

Expected Results	Data stored within 2 seconds
Actual Result	Data saved in 1.5 seconds
Remarks	Pass

Table 84:Performance Test Case 3

5.4.3 Test Case 3

Testcase ID	PT3
Title	Reminder Notification Performance
Description	This test verifies that scheduled healthcare reminders are triggered at the specified time.
Objective	To ensure that reminder notifications are generated and displayed on time.
Driver/precondition	User is logged in and a reminder has been scheduled.
Test steps	1.Create reminder 2. Set reminder time 3. Wait for scheduled time 4.Observe notification.
Input	Reminder details and scheduled time
Expected Results	Notification should be triggered at or close to the scheduled time without significant delay.
Actual Result	Reminder notification was triggered at the scheduled time.
Remarks	Pass

Table 85:Performance Test Case 4

5.4.4 Test Case 4

Testcase ID	PT4
Title	Chat Performance

Description	This test verifies the response time of sending and receiving messages between the patient and doctor.
Objective	To ensure that chat messages are sent, received, and displayed within an acceptable response time.
Driver/precondition	Patient and doctor are logged in, and the chat feature is available after approval.
Test steps	1.Open chat 2. Patient sends a message and Observe message delivery 4. Doctor receives the message and sends a reply 6. Patient receives the reply 7. Observe response time.
Input	Chat messages from patient and doctor
Expected Results	Messages should be sent,receive and displayed without significant delay.
Actual Result	Messages were successfully sent and received within an acceptable response time.
Remarks	Pass

Table 86:Performance Test Case 5

5.4.5 Test Case 5

Testcase ID	PT5
Title	Chatbot Response Time
Description	Tests how quickly chatbot responds to user queries.
Objective	Ensure real-time response for better user experience.
Driver/precondition	Internet connection active
Test steps	1. Open chatbot

	2. Enter query 3. Measure response time
Input	Health-related query
Expected Results	Response within 2–4 seconds
Actual Result	Response received in 3 seconds
Remarks	Pass

Table 87:Performance Test Case 6

5.4.6 Test Case 6

Testcase ID	PT6
Requirement ID	R6 (System Load Handling)
Title	Multiple Users Load Test
Description	Tests system performance when multiple users access simultaneously.
Objective	Ensure the system remains stable under load.
Driver/precondition	Multiple users simulated
Test steps	1. Simulate multiple users 2. Perform actions (login, add data) 3. Monitor performance
Input	Multiple user requests
Expected Results	System remains stable without crash
Actual Result	System handled load successfully
Remarks	Pass

Summary

This chapter focuses on testing and evaluation of our project, Unit testing, Integration testing, functional and performance testing of several units and components were conducted with satisfactory results.

Chapter 6
System Conversion

6.1 Conversion Method

For the implementation of the Smart Healthcare Reminder the system conversion method should ensure that the system should work smoothly from the old reminder system to the new intelligent system.

6.1.1 Pilot Conversion

In the pilot conversion approach the new system is initially implemented in the limited controlled environment for limited users or one department of hospital. This allows for comprehensive testing of the system's core functionalities, identification of real-world issues and collection of user feedback before full-scale deployment.

Justification:

Reduced Risk: Since the limited user used the system initially the risk of system wide failure is minimized.

Focused Testing: Allow in depth testing of modules user authentication, medication, measurement, activity, measurement tracker, track intake ,appointment and notification system.

User Feedback Loop: Early user feedback help to improve the App performance and usability before wider implementation.

Pilot Scope:

Initial Group: A selected group of patient will use the Smart Healthcare Reminder application at the initial stage.

Modules Tested Authentication, Medication, Measurement, Activity, Measurement tracker, track intake, Appointment reminders and doctor, caregiver or pharmacist dashboard.

Duration: Two week of controlled usage and evaluating system performance, identifying issues and making necessary improvements the system will be fully deployed for all users.

6.2 Deployment

Deployment involves preparing and transferring the Smart Healthcare Reminder System from the development environment to a live environment where it can be used by patients, doctors, and caregivers. This process ensures that the system is fully functional, secure, and accessible to end users. This includes building the application, configuring Firebase services, setting up secure access, and ensuring the application runs successfully on the users' Android devices.

6.2.1 Deployment steps:

Step No.	Task	Details
1	Build APK/Bundle	Build signed APK using Android Studio for production deployment.
2	Backend Configuration	Ensure Firebase Firestore, Realtime Database, and Firebase Auth are live.
3	Firebase Rule Setup	Apply role based read/write security rules for Firestore and Database.
4	App Deployment	Distribute APK through selected users such as patient ,doctor , caregiver, pharmacist via secure Google play, WhatsApp.
5	Device Testing	Install various Android versions for compatibility testing.
6	Live Environment Verification	Check connectivity with Firebase and the proper working of reminders and notification systems.
7	Deployment Confirmation	Conduct user acceptance testing with limited patients for validation.

6.2.2 Data Conversion

Step	Task	Details
1	Data Preparation	Collect and organize healthcare-related data such as patient details, medication records, measurement types (BP, sugar, weight), activity types (water intake, walking, meals, exercise, yoga, cycling),doctor profiles, and appointment information.
2	Data Cleaning	Remove duplicate entries, incorrect values (e.g., invalid BP/sugar readings), incomplete patient records and fix formatting issues before storing data.

3	Data Transformation	Convert collected data into structured format suitable for Firestore collections such as Users, Medications, Measurements, Activities, Appointments, and Chat,user history
4	Data Loading	Upload data into Firebase Firestore using Firebase console . Ensure proper mapping between patient, doctor, and caregiver data.

6.2.3 Training

For training purposes a descriptive video is created to guide users how to use Smart Healthcare Reminder Application.The video demonstrates the real usage scenarios for different actors such as patient, doctor, caregiver and pharmacist

Step	Instruction
1	Open the application and login using email and password or sign up for a new account.
2	Enter personal details such as name, age, gender, Date of birth during registration.
3	After login, access the dashboard/home screen.
4	Navigate through different modules (Medication, Measurement, Activity, Appointment).
5	Customize notification preferences and profile settings.

Step	Instruction
1	Open the Medication module from the dashboard.
2	Add new medicine by entering name, dosage, and time.
3	Save the medication to activate the reminder.
4	Receive notification at scheduled time.
5	Mark medicine as “Taken” or “Missed”.

Step	Instruction
1	Navigate to the Measurement module.
2	Enter health data such as BP, sugar level, or weight.
3	Save and track history of measurements.

4	Open Activity module to set reminders (water, walking, meals).
5	Receive notifications and update activity status.

Step	Instruction
1	Open Appointment module.
2	Select the doctor and choose date/time.
3	Send request to doctor through email by giving information and accept the doctor.
4	Doctor views the patient profile and health data.
5	Doctor provides recommendations through the system.

Step	Instruction
1	Assign or connect caregiver to patient account.
2	Caregiver logs in and views patient data.
3	Caregiver updates reminder status if needed.
4	Add emergency contact details in profile section.
5	Access emergency contact quickly when required.

6.3 Post Deployment Testing

Post Deployment testing was conducted to validate that the Smart Healthcare Reminder System is properly installed, configured and operating as intended. The objective was to ensure data integrity, feature functionality and user experience after deployment.

6.3.1 Post Deployment Testing Steps:

Steps	Test Activity	Expected Outcomes
1	Launch Application	Application starts successfully without crashes or errors.
2	Login Functionality	User can login and signup using Email and password.
3	Medication Reminder	Users can add medication and receive timely notifications.
4	Measurement Tracking	Users can record BP, sugar, and weight and view history correctly.
5	Activity Reminder	Daily activities (water, meals, exercise, walking, yoga or cycling) reminders are triggered properly.

6	Data Integrity and Verification	All user data (medication, measurements, activities) remains consistent after login/logout.
7	Notification system	Notifications are triggered at correct time.
9	Performance Check	Application performs smoothly with acceptable response time on different devices.

6.4 Challenges and Solutions

During deployment and post deployment phases of Smart Healthcare Reminder System several technical and operational challenges were encountered. Below is the list of main challenges along with their applied solution.

Challenge	Description	Solution Implemented
Firebase Data Synchronization Issues	Delay or inconsistency in syncing patient data (medication, measurement, reminders)across modules.	Implemented real-time listeners and optimized Firestore queries to ensure instant data synchronization.
Notification Delay or Failure	Some reminders were not triggered on time due to background restrictions.	Used reliable scheduling techniques (Alarm Manager) and optimized notification handling.
Role-Based Access Control (Patient/Doctor/Caregiver/Pharmacist)	Incorrect permission causes unauthorizes access or restricted features.	Defined and tested proper Firestore security rules for each role to ensure secure access.
APK Installation Issues	Some devices blocked installation of unsigned APK files.	Generated signed APK using Android Studio and distributed securely.
Data Entry Errors (Health Records)	Users entered incorrect or incomplete health data (e.g., invalid BP or sugar values).	Added input validation and error messages to ensure correct data entry.
Device Compatibility Issues	Uilayout issues on different screen sizes and older Android devices.	Used responsive UI design and tested on multiple emulators (API 24+) and real devices.
Internet Dependency	Application features depends on internet connectivity causing issues on offline mode.	Added user notifications for connectivity status and minimized data usage where possible
Chatbot Response Accuracy	Chatbot sometimes gave general or unclear responses.	Improved chatbot logic and limited responses to basic, safe health guidance.

Multiple Reminder Overload	Too many notifications confused users.	Implemented reminder grouping and limited repeated alerts to improve usability.
Caregiver Linking Issues	Difficulty in linking caregiver with patient account.	Simplified linking process and added clear instructions in UI
Security & Privacy Concerns	Sensitive health data needed protection.	Applied Firebase Authentication and secured Firestore rules to protect user data.

6.5 Summary

This chapter focuses on how our app is deployed and what training does user needs, how we convert data into information that is useful for our project .Also it focuses on challenges faced during development and how these challenges were tackled.

Chapter 7
Conclusion

7. Introduction

This chapter will conclude what has been completed, partial, or still under development.

Objectives	Status
User Sign Up and Login with email and password	Completed
Medication Reminders with notifications	Completed
Measurement Reminders (BP, Sugar, Weight)	Completed
Activity Reminders (Water, Meal, Walk, Exercise)	Completed
Appointment Reminders with doctor name	Completed
Medicine Refill Reminders	Completed
Health Measurement Tracker	Completed
Track Intake (Taken / Missed status)	Completed
Pharmacy for medicine delivery	Completed
AI Chatbot for health questions	Completed
Doctor Module to view patient data	Completed
Caregiver Module to monitor patient	Completed
Profile Section with history and feedback	Completed
Admin manages users and feedback	Completed

7.1 Conclusion

Conclusion drawn from our work and results is that it was the need of the hour and users accepted the app and used it and liked it.

7.2 Future Works

After completing this project, we plan to develop a Web and iOS version of the app. More features like online doctor consultation and advanced AI health suggestions will also be added.

References:

- National Library of Medicine. (2023).
- Medisafe Pill & Med Reminder (2025). Medisafe Pill & Medication Reminder.
- Dosecast (2025). Dosecast: Medication Reminder & Tracker https://play.google.com

- Google Developers (2026). Android Developers https://developer.android.com
- Flutter Developers. (2026). Flutter Documentation. http://docs.flutter.dev
- Dart Team. (2026). Dart Programming Language. http://dart.dev
- Firebase. (2026a). Cloud Firestore. http://firebase.google.com/docs/firestore
- Firebase. (2026b). Firebase Authentication. http://firebase.google.com/docs/a
- Firebase. (2026c). Firebase Cloud Messaging. http://firebase.google.com/docs/cloud-messaging

Appendix A: General Coding Standards & Guidelines

To ensure the maintainability, readability, and scalability of the codebase for Medicare – A Smart Healthcare Reminder App, the following coding standards and guidelines were followed throughout the development using Flutter and Firebase.

1. Variable Naming Conventions

Consistent naming is used throughout the app.

The following conventions are used based on the context:

- snake_case: Used for file names and folder names (e.g., medication_screen.dart).
- PascalCase: Used for class and widget names (e.g., MedicationReminder, HomeScreen).
- camelCase: Used for variables and function names (e.g., reminderTime, saveReminder()).

2. Visual Scope Indicators

Variables are named clearly to show their scope:

- PrivateVariable (underscore means it is private in Dart)
- const MAX_RETRIES = 3
- final String userId

3. Read-Only and Immutable Fields

- The final keyword is used for values that do not change after being set.
- const keyword i
-
- s used for values that are fixed at compile time.
- Example: final String userId = auth.currentUser!uid.

4. Function Naming and Behavior

All function names clearly describe what they do:

- saveMedicationReminder()
- loadUserData()
- scheduleNotification()
- markIntakeAsTaken()

- fetchPatientRecords()

5. Commenting Style

- Comments are added where the code is not easy to understand.

Example:

```
// This function saves the medication reminder to Firestore
Future<void> saveMedicationReminder(Reminder reminder) async {
  // Validate fields before saving
}
```

6. Indentation and Code Structure

- 2-space indentation is used as per Flutter/Dart standard.
- Each widget and function is clearly separated for easy reading.
- Related code is grouped inside the same file or class.

7. File and Folder Naming

All file names use snake_case as required by Dart:

- medication_screen.dart
- notification_helper.dart
- chatbot_screen.dart
- Folders are organized by feature:
- auth/medication/measurement/ activity/appointment/chatbot/ admin.

8. Widget and Class Structure

- Each widget or class handles only one responsibility.
- UI widgets are separated from business logic.
- Provider or setState is used for state management.
- Firebase calls are handled inside repository classes, not inside UI widgets.

9. Access Modifiers and Scoping

- Private variables use underscore prefix (e.g. reminderTime).
- Public methods are only made public when needed by another class.
- Local variables are declared inside functions to limit their scope.

10. Cohesion and Coupling

- Each module (Medication, Measurement, Activity, etc.) works independently.
- Shared utilities like NotificationHelper and FirebaseHelper are used across modules without tight dependency.

11. DRY Principle (Don't Repeat Yourself)

- Common logic is written once and reused across modules.
- Helper classes like FirebaseHelper and InputValidator are shared across the whole app.
- Repeated UI components are made into reusable widgets.

12. Single Responsibility Principle (SRP)

- Each part of the code has one job.
- Function handles a single task.
- Class handles a single responsibility (e.g., Reminder Scheduler, UserAuth).
- Screen file only handles the UI for that screen.

13. Extensibility and Maintainability

- New modules can be added without changing existing code.
- Shared base classes and utility functions make future updates easy.
- Firebase structure is designed to support adding new features easily.

14. Exception Handling and Logging

- All Firebase and network calls are wrapped in try-catch blocks.
- Errors are shown to the user with simple error messages using SnackBar.

Example:

```
try {
  await firestore.collection('reminders').add(data);
} catch (e) {
  print('Failed to save reminder: $e');
}
```